UNIVERSIDADE FEDERAL DE SANTA MARIA
CENTRO DE TECNOLOGIA
CURSO DE BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO

Murilo Santos Leal

PLATAFORMA WEB PARA AVALIAÇÃO COMPARATIVA DE MODELOS DE IA EM CONSULTAS EM LINGUAGEM NATURAL A BANCOS DE DADOS RELACIONAIS

Santa Maria, RS
2025

Murilo Santos Leal

PLATAFORMA WEB PARA AVALIAÇÃO COMPARATIVA DE MODELOS DE IA EM CONSULTAS EM LINGUAGEM NATURAL A BANCOS DE DADOS RELACIONAIS

Trabalho Final de Graduação apresentado ao Curso de Bacharelado em Ciência da Computação da Universidade Federal de Santa Maria (UFSM, RS), como requisito parcial para obtenção do grau de **Bacharel em Ciência da Computação**.

ORIENTADORA: Profa.Andrea Schwertner Charão

Santa Maria, RS
2025

Murilo Santos Leal

PLATAFORMA WEB PARA AVALIAÇÃO COMPARATIVA DE MODELOS DE IA EM CONSULTAS EM LINGUAGEM NATURAL A BANCOS DE DADOS RELACIONAIS

Trabalho Final de Graduação apresentado ao Curso de Bacharelado em Ciência da Computação da Universidade Federal de Santa Maria (UFSM, RS), como requisito parcial para obtenção do grau de **Bacharel em Ciência da Computação**.

Aprovado em 28 de novembro de 2025:

Andrea Schwertner Charão (UFSM)
(Presidente/Orientadora)

Célio Trois (UFSM)

Fernando Pires Barbosa (UFSM)

Santa Maria, RS
2025

AGRADECIMENTOS

Agradeço, antes de tudo, à minha namorada Francieli, por sua presença constante, pelo carinho diariamente renovado e pela paciência nos momentos em que o cansaço parecia maior que o caminho. Seu apoio foi o que manteve este trabalho possível e leve, mesmo quando os desafios se acumulavam.

Aos meus pais, Marcelo e Rozaida, deixo minha gratidão mais profunda. Foram eles que me ensinaram, desde cedo, o valor do estudo, da honestidade e da persistência. Tudo o que conquistei na universidade tem raízes no esforço e nos princípios que recebi em casa.

À minha irmã, Marceli, que sempre acreditou em mim e comemorou cada pequena vitória deste percurso, mesmo quando eu próprio não enxergava o progresso. Obrigado por ser apoio, companhia e inspiração constante.

Estendo também minha homenagem aos meus avós e avôs, Arlinda e Lorelei, que acompanharam esta jornada com orgulho e carinho, e aos que partiram, João e Antônio Manoel (Téo), cuja memória permanece viva em mim e segue iluminando minhas escolhas. Suas histórias, conselhos e afetos me trouxeram até aqui.

De maneira especial, agradeço ao meu tio Paulo Sidnei (Neide), cuja partida ocorreu ainda durante minha trajetória na universidade. Sua presença marcou minha infância e adolescência, e seus ensinamentos seguem presentes em cada decisão e em cada conquista. Este trabalho também é para ele.

Expresso ainda meu sincero agradecimento à minha orientadora, Andrea, pela orientação cuidadosa, pelas sugestões técnicas, pela paciência nas revisões e pela liberdade que me concedeu para explorar ideias e amadurecer a pesquisa. Sua presença foi essencial para que este trabalho alcançasse solidez acadêmica.

Agradeço também aos professores e profissionais que compõem a banca avaliadora. Suas leituras atentas e contribuições críticas certamente enriquecerão este trabalho e minha formação.

Aos meus colegas de trabalho e amigos que compartilharam conversas, ideias, risadas e incentivos ao longo do desenvolvimento deste projeto, deixo minha gratidão. O entusiasmo de vocês ajudou a transformar uma ideia em uma aplicação real e relevante.

Por fim, registro minha gratidão a todos que, de alguma forma, caminharam ao meu lado, professores, colegas de curso e pessoas que torceram por mim. Cada gesto de apoio, por menor que parecesse, contribuiu para que este trabalho se concretizasse. A todos, meu sincero muito obrigado.

"Se ninguém mais se lembrar de nós, isso não significa que não existimos."

Dark (Netflix)

PLATAFORMA WEB PARA AVALIAÇÃO COMPARATIVA DE MODELOS DE IA EM CONSULTAS EM LINGUAGEM NATURAL A BANCOS DE DADOS RELACIONAIS

WEB PLATFORM FOR COMPARATIVE EVALUATION OF AI MODELS IN NATURAL LANGUAGE QUERIES TO RELATIONAL DATABASES

Murilo Santos Leal[1], **Andrea Schwertner Charão**[2]

RESUMO

Este trabalho apresenta o desenvolvimento do *QueryLab*, uma plataforma web voltada à avaliação comparativa de modelos de Inteligência Artificial capazes de traduzir perguntas em linguagem natural para consultas SQL executáveis sobre bases relacionais. A solução parte do desafio enfrentado por usuários não especialistas ao formular consultas complexas e da necessidade de analisar, de forma transparente e reproduzível, o desempenho de diferentes modelos generativos nessa tarefa. A plataforma integra três modelos gratuitos acessíveis via API, Gemini, Groq (Llama 3.3 70B) e SQLCoder, executando-os em paralelo para uma mesma pergunta e produzindo múltiplas hipóteses de SQL. Cada hipótese é validada sintaticamente, executada sobre dados públicos do INEP e retornada ao usuário acompanhada da consulta final, de uma explicação em linguagem natural, de métricas de execução (tempo, linhas retornadas, erros) e de indicadores de confiabilidade. A interface permite a comparação lado a lado das respostas, favorecendo análises mais críticas sobre as diferenças entre modelos. A avaliação exploratória, realizada com estudantes e profissionais das áreas de Computação e Planejamento da Universidade Federal de Santa Maria, indicou que a estratégia paralela facilita a comparação entre modelos, aumenta a confiança do usuário na interpretação das respostas e evidencia limitações importantes da geração automática de SQL, especialmente em cenários de esquemas extensos e consultas mais complexas. Esses achados apontam oportunidades para pesquisas futuras envolvendo explicabilidade, técnicas especializadas de NL2SQL e estratégias de avaliação contínua.

Palavras-chave: Inteligência Artificial. Linguagem Natural. SQL. Modelos Generativos. Bancos de Dados Relacionais. Interfaces Conversacionais.

ABSTRACT

This work presents the development of QueryLab*, a web platform for the comparative evaluation of Artificial Intelligence models capable of translating natural language questions into executable SQL queries over relational databases. The system addresses the challenges faced by non-expert users when formulating complex SQL statements and the need for transparent and reproducible assessment of generative models in this task. The platform integrates three free API-based models—Gemini, Groq (Llama 3.3 70B), and SQLCoder—executing them in parallel for each query and generating multiple SQL hypotheses. Each hypothesis is syntactically validated, executed on*

[1] Autor: Graduando no Curso de Bacharelado em Ciência da Computação pela Universidade Federal de Santa Maria.

[2] Orientadora: Professora do Departamento de Linguagens e Sistemas de Computação da Universidade Federal de Santa Maria.

public INEP educational data, and returned with the final SQL command, a natural-language explanation, execution metrics (such as runtime, returned rows, and errors), and confidence indicators. The interface enables side-by-side comparison of results, supporting more critical analysis of model behavior. An exploratory evaluation conducted with students and professionals from the Computing and Planning sectors at the Federal University of Santa Maria indicated that the parallel strategy improves model comparison, increases user confidence in interpreting responses, and highlights relevant limitations of automatic SQL generation, especially in scenarios involving large schemas and more complex queries. These findings point to opportunities for future research involving explainability, specialized NL2SQL techniques, and continuous evaluation strategies.

***Keywords**: Artificial Intelligence. Natural Language. SQL. Generative Models. Relational Databases. Conversational Interfaces.*

LISTA DE FIGURAS

Figura 1 – Diagrama de sequência do *QueryLab*, com separação entre frontend, backend, camada de orquestração e bancos de dados. 27
Figura 2 – Interface de comparação paralela entre Gemini, Groq e SQLCoder para uma mesma pergunta. 30
Figura 3 – Tela inicial do *QueryLab*, com mensagens e cards explicativos sobre o produto, seleção de modelos, tema, informações de conexão e histórico de sessões. 31
Figura 4 – Visão técnica da execução, com SQL gerada, tempo de execução, linhas retornadas e amostra de dados. 32
Figura 5 – Visão em formato de chat, com a resposta em linguagem natural para a hipótese selecionada. 33
Figura 6 – Tela de avaliação da qualidade das consultas SQL geradas, com notas atribuídas pelos usuários em diferentes dimensões. 34
Figura 7 – Erro de execução com diagnóstico automático e sugestão de refinamento. 36
Figura 8 – Tela de avaliação geral, com consolidação de métricas objetivas e qualitativas coletadas durante o uso. 37

LISTA DE TABELAS

Tabela 1 – Síntese de abordagens em trabalhos correlatos e implicações práticas. . 19
Tabela 2 – Comparativo entre principais *benchmarks* de *text-to-SQL*. 22
Tabela 3 – Comparativo detalhado entre os estudos da UFSM e o presente projeto. 24
Tabela 4 – Principais lacunas e oportunidades identificadas nos trabalhos da UFSM. 24
Tabela 5 – Componentes principais da arquitetura do sistema *QueryLab*. 27
Tabela 6 – Regras de validação aplicadas às consultas geradas pelos modelos. . . 35

LISTA DE SIGLAS

API Application Programming Interface

BD Banco de Dados

CAPES Coordenação de Aperfeiçoamento de Pessoal de Nível Superior

CoSQL Conversational Text-to-SQL Benchmark

CT Centro de Tecnologia

DAIL-SQL Data-Aware In-Context Learning SQL

DDL Data Definition Language

IA Inteligência Artificial

IBGE Instituto Brasileiro de Geografia e Estatística

INEP Instituto Nacional de Estudos e Pesquisas Educacionais Anísio Teixeira

IES Instituições de Ensino Superior

LLM Large Language Model

NL2SQL Natural Language to SQL

NLIDB Natural Language Interface to Databases

PPG Programa de Pós-Graduação

SQL Structured Query Language

SParC Sequential Text-to-SQL Benchmark

TCC Trabalho de Conclusão de Curso

UF Unidade Federativa

UFSM Universidade Federal de Santa Maria

WCAG Web Content Accessibility Guidelines

SUMÁRIO

Lista de Siglas **10**
1 Introdução 13
2 Fundamentação e Trabalhos Relacionados 14
2.1 Problema das consultas em linguagem natural e evolução do *text-to-SQL* 15
2.1.1 Exemplo 1: contagem de instituições por unidade federativa 15
2.1.2 Exemplo 2: matrículas em faixa etária específica por município 16
2.1.3 Exemplo 3: evolução temporal de cursos na área de Computação 17
2.2 LLMs, orquestração da inferência e estratégias para *text-to-SQL* . 17
2.2.1 DAIL-SQL: contexto como variável de primeira ordem . 18
2.2.2 SQLGenie: pipeline industrial com múltiplas inferências e controle 18
2.2.3 SDE-SQL: sondagens intermediárias e exploração orientada à execução 19
2.2.4 Síntese de abordagens e implicações práticas 19
2.2.5 Estratégias de *prompt engineering*, pré-processamento e pós-checagem 20
2.3 Avaliação, *benchmarks* e limites das métricas tradicionais 20
2.4 Ferramentas relacionadas de observabilidade e avaliação 22
2.5 Trabalhos correlatos na UFSM, lacunas e posicionamento deste trabalho 23
3 Metodologia, Projeto e Implementação 25
3.1 Metodologia geral e requisitos da plataforma 25
3.2 Visão geral do sistema e arquitetura em camadas 26
3.3 Fluxo de orquestração entre usuário, modelos e banco de dados . . 28
3.4 Evolução arquitetural e método paralelo 30
3.5 Implementação do frontend 30
3.6 Backend, segurança e validação das consultas SQL 35
3.7 Avaliação, métricas e coleta de dados 36
3.8 Síntese da metodologia 38
4 Resultados e Avaliação 38
4.1 Objetivos de avaliação e hipóteses 38
4.2 Dados utilizados e caracterização do esquema 38
4.3 Ambiente experimental 39
4.4 Métricas e instrumentos 40
4.5 Procedimento 41
4.6 Resultados 41
4.7 Avaliação qualitativa via formulário 41
4.8 Discussão 42
5 Conclusão 42

Referências **45**
APÊNDICE A Documentação técnica do esquema INEP e do processo de descoberta **47**
1 Script de descoberta do esquema 47
2 Resumo condensado do esquema 48
3 Documentação textual do esquema 48
4 Estrutura completa de metadados 49
5 Disponibilização integral dos arquivos 49
APÊNDICE B Algoritmos Auxiliares do *QueryLab* **51**

1 INTRODUÇÃO

Nas últimas décadas, o avanço da Inteligência Artificial (IA) tem transformado de maneira profunda a forma como interagimos com sistemas computacionais. Ferramentas baseadas em modelos de linguagem natural, como assistentes virtuais e plataformas de análise de dados, tornaram-se soluções amplamente acessíveis e capazes de apoiar decisões em diferentes contextos. Segundo o relatório *Predicts 2025* da Gartner, a IA generativa deverá provocar disrupções significativas nos mercados de gestão de dados, impulsionando a adoção de interfaces que permitem formular consultas e explorar informações diretamente em linguagem natural, sem a necessidade de conhecimento técnico avançado (Zaidi et al., 2024). Esse movimento reflete uma tendência global de tornar o acesso a dados mais intuitivo e inclusivo.

Apesar dessas transformações, a linguagem SQL permanece como padrão de fato para a manipulação de dados relacionais e continua essencial em sistemas corporativos, governamentais e acadêmicos. Contudo, escrever consultas SQL complexas, envolvendo múltiplas tabelas, filtros compostos, agregações e cláusulas aninhadas, constitui uma barreira significativa, especialmente para profissionais sem formação técnica. Mesmo especialistas enfrentam desafios em esquemas extensos ou heterogêneos, nos quais a compreensão estrutural do banco de dados exige esforço considerável (Quamar et al., 2022; Deng; Chen; Zhang, 2022). Assim, cresce a demanda por soluções que reduzam essa complexidade e ampliem o acesso qualificado a informações.

Nesse contexto, surgem as interfaces de linguagem natural para bancos de dados (NLIDB, sigla para Natural Language Interfaces to Databases), cujo objetivo é permitir que usuários consultem bases relacionais utilizando sentenças em linguagem natural. As primeiras abordagens eram fundamentadas em regras linguísticas e gramáticas formais, mas, com o avanço do aprendizado profundo, modelos neurais passaram a dominar o campo. Mais recentemente, modelos de linguagem de grande porte (LLMs) ampliaram a capacidade de generalização e cobertura dessas interfaces, embora tragam desafios relacionados a custo, latência, explicabilidade e avaliação transparente (Yu et al., 2018; Scholak; Schucher; Bahdanau, 2021).

A avaliação desses sistemas é um aspecto crítico. Diferentes modelos podem gerar consultas SQL estruturalmente distintas para uma mesma pergunta, variando em correção sintática, adequação semântica, eficiência de execução e clareza da explicação. Métricas tradicionais, como *exact match* ou acurácia de execução, capturam apenas parte desse comportamento, como amplamente discutido nos benchmarks Spider e CoSQL, que evidenciam limitações dessas métricas diante da diversidade de esquemas e da complexidade semântica das consultas (Yu et al., 2018; Yu et al., 2019). Além disso, estudos sobre NLIDBs mostram que, em cenários reais, especialmente em bancos de dados amplos e heterogêneos, diferentes modelos podem produzir consultas plausíveis, porém incorretas ou inconsistentes, refletindo desafios de robustez, alinhamento semântico e generalização (Quamar et al., 2022; Zhang, 2024). Assim, torna-se necessário adotar mecanismos de avaliação mais abrangentes, que combinem métricas automáticas, telemetria e julgamento humano, alinhando-se a tendências recentes de avaliação contínua e cenários evolutivos presentes em iniciativas como o LiveSQLBench (Team, 2024).

A relevância desse problema é especialmente evidente em domínios públicos, como educação, saúde ou mobilidade urbana, nos quais bases extensas e de grande impacto social precisam ser acessadas por profissionais com formações variadas. No

caso brasileiro, destacam-se os microdados abertos do Instituto Nacional de Estudos e Pesquisas Educacionais Anísio Teixeira (INEP), que reúnem informações detalhadas sobre instituições, estudantes e cursos em escala nacional. Embora esses dados sejam valiosos para políticas públicas e pesquisas acadêmicas, seu uso efetivo exige conhecimento técnico especializado em SQL e arquitetura de dados, uma limitação já destacada na literatura sobre interfaces de linguagem natural para bancos de dados e no contexto de dados educacionais brasileiros (Quamar et al., 2022; Trevisan, 2024; Assumpção, 2025). Tecnologias capazes de traduzir perguntas em linguagem natural para SQL, acompanhadas de resultados explicados e avaliáveis, podem reduzir essa barreira e ampliar de maneira significativa o impacto dos dados educacionais abertos, alinhando-se às tendências de ampliação do acesso a dados impulsionadas por sistemas baseados em IA (Zaidi et al., 2024).

Diante desse cenário, este trabalho tem como objetivo desenvolver uma plataforma que possibilite formular consultas em linguagem natural, gerar múltiplas hipóteses de SQL com diferentes modelos de linguagem e avaliar seu desempenho em condições reais de uso. O sistema proposto adota uma arquitetura híbrida, com múltiplos provedores de LLM, mecanismos de *fallback*, validação sintática, execução segura e telemetria detalhada. Além disso, incorpora avaliação humana estruturada, permitindo que o usuário julgue dimensões como clareza, correção e utilidade das respostas, complementando as métricas automatizadas.

A contribuição deste trabalho está na integração de técnicas modernas de *text-to-SQL* com uma plataforma interativa que funciona simultaneamente como ferramenta de apoio à exploração de dados e como laboratório de experimentação. Ao combinar múltiplos modelos, explicações em linguagem natural, histórico de execuções e métricas objetivas, o sistema permite analisar divergências entre modelos, entender limitações e identificar padrões de comportamento que seriam difíceis de observar manualmente. Dessa forma, esta pesquisa avança o estado da arte institucional, oferecendo uma solução alinhada com os desafios de confiabilidade, explicabilidade e avaliação discutidos na literatura.

O restante deste texto está organizado da seguinte forma: na Seção 2, apresenta-se a fundamentação teórica sobre bancos de dados relacionais, consultas em linguagem natural e técnicas *text-to-SQL*. A Seção 3 descreve a metodologia adotada, incluindo arquitetura, orquestração entre modelos e implementação do sistema. Na Seção 4, são apresentados os experimentos realizados e a análise dos resultados. Por fim, a Seção 5 apresenta as conclusões e propõe direções para trabalhos futuros.

2 FUNDAMENTAÇÃO E TRABALHOS RELACIONADOS

Esta seção apresenta uma revisão técnica da tarefa *text-to-SQL* e dos trabalhos relacionados, organizada em quatro eixos principais: (i) a problemática das consultas em linguagem natural e a evolução histórica da área; (ii) a mudança de paradigma trazida pelos modelos de linguagem de grande porte (LLMs) e as estratégias de orquestração da inferência; (iii) os desafios de avaliação, *benchmarks* e métricas; e (iv) os trabalhos correlatos no contexto da Universidade Federal de Santa Maria (UFSM) e as lacunas que motivam este projeto.

2.1 Problema das consultas em linguagem natural e evolução do *text-to-SQL*

A tarefa *text-to-SQL* visa traduzir uma consulta em linguagem natural para um comando SQL executável em bancos de dados relacionais. O problema insere-se no campo de *semantic parsing*, isto é, o mapeamento de expressões textuais para estruturas formais (Quamar et al., 2022). Historicamente, os primeiros sistemas de *text-to-SQL* baseavam-se em gramáticas definidas manualmente, regras de mapeamento e abordagens estatísticas tradicionais. Com o avanço do aprendizado profundo, arquiteturas *encoder–decoder* passaram a dominar esse domínio, culminando em modelos como Seq2SQL (Zhong; Xiong; Socher, 2017), SQLNet (Xu; Liu; Song, 2017) e, posteriormente, sistemas baseados em *transformers* capazes de lidar com múltiplas tabelas, chaves estrangeiras e consultas complexas com agregações, filtros compostos e subconsultas (Deng; Chen; Zhang, 2022; Hong et al., 2024). Esses modelos representam um progresso significativo, mas ainda dependem fortemente do conhecimento explícito do esquema para gerar SQL correto.

A literatura recente identifica dois movimentos complementares. De um lado, há o refinamento de arquiteturas especializadas para *text-to-SQL*, com ênfase em técnicas de *schema linking*, pré-treinamento em *corpora* tabulares e decodificação restrita. Exemplos incluem o RAT-SQL, que codifica explicitamente relações entre colunas, tabelas e valores (Wang et al., 2020), e o PICARD, que introduz um mecanismo de decodificação incrementalmente verificada para garantir que a saída permaneça sintaticamente válida em SQL (Scholak; Schucher; Bahdanau, 2021). De outro lado, surgem abordagens que exploram modelos de linguagem de propósito geral (LLMs) como motores de geração de consultas, eventualmente combinando-os com verificadores de consistência, exemplos em contexto ou esquemas anotados (Hong et al., 2025; Gao et al., 2024; Ghosh; Jain; Yenigalla, 2025). Em ambos os casos, a qualidade do SQL gerado depende de forma crítica de como o modelo acessa (ou é condicionado por) o esquema real do banco de dados.

Antes de apresentar os exemplos a seguir, é importante contextualizar o leitor quanto às tabelas mencionadas. Todos os casos utilizam o esquema relacional real adotado neste trabalho, derivado das bases públicas do INEP, cujo resumo técnico e documentação encontram-se no Apêndice A. Esse esquema inclui tabelas como `censo_ies`, `censo_ies_bruto`, `matriculas_municipio`, `municipios_ibge` e `capes_cursos_bruto`, entre outras, utilizadas para representar instituições, cursos, matrículas e municípios. Assim, as consultas SQL ilustradas não são hipotéticas: elas refletem a estrutura concreta dos dados analisados e servem para demonstrar como um modelo de linguagem depende da correta compreensão desse esquema para produzir SQL executável.

Para evidenciar essa dependência, é útil observar como uma mesma pergunta em linguagem natural pode resultar em respostas bastante diferentes a depender de o modelo ter, ou não, acesso ao esquema do banco de dados. A seguir, são discutidos três exemplos baseados no esquema real utilizado neste trabalho, derivado de bases públicas do INEP/CAPES, que incluem informações sobre instituições de ensino superior, cursos, matrículas e municípios. Em cada exemplo, contrasta-se um SQL ideal, isto é, uma consulta consistente com o esquema de fato, com a tendência de resposta de um LLM genérico que apenas "alucina" valores a partir de conhecimento textual.

2.1.1 Exemplo 1: contagem de instituições por unidade federativa

Considere a pergunta em linguagem natural:

"Quantas instituições de ensino superior existem no estado de Minas Gerais?"

No esquema adotado, as instituições aparecem na tabela `censo_ies`, enquanto informações de localização (UF e município) são detalhadas na tabela `censo_ies_bruto`, que contém a sigla da unidade federativa no atributo `sg_uf_ies`. Uma formulação de SQL consistente com esse esquema seria:

```
SELECT COUNT(DISTINCT i.cod_ies) AS total_ies_mg
FROM censo_ies AS i
JOIN censo_ies_bruto AS b
  ON b.co_ies = i.cod_ies
WHERE b.sg_uf_ies = 'MG';
```

Essa consulta utiliza a chave institucional (`cod_ies`/`co_ies`) para integrar as duas tabelas, filtra explicitamente a UF de interesse e aplica `COUNT(DISTINCT ...)` para contabilizar cada instituição apenas uma vez, mesmo que apareça em múltiplos registros agregados. Um modelo de linguagem sem acesso ao banco de dados, em contraste, tenderia a responder com um texto genérico, por exemplo: "Minas Gerais possui dezenas de instituições de ensino superior, incluindo UFMG, UFOP, UFU e diversas faculdades privadas", sem fornecer um número exato, pois não tem como executar a contagem sobre os dados reais.

2.1.2 Exemplo 2: matrículas em faixa etária específica por município

Como segundo exemplo, considere uma pergunta que combina recorte territorial e demográfico:

"Quantas matrículas existem na faixa etária de 18 a 24 anos no município de Santa Maria (RS) no último Censo da Educação Superior?"

Neste caso, o esquema dispõe de uma tabela agregada por município, `matriculas_municipio`, com colunas que já representam faixas etárias (por exemplo, `qt_mat_18_24`), e de uma tabela de referência de municípios, `municipios_ibge`, contendo o nome do município, o código IBGE e a UF. Uma formulação plausível de SQL seria:

```
SELECT SUM(m.qt_mat_18_24) AS total_matriculas_18_24
FROM matriculas_municipio AS m
JOIN municipios_ibge AS mu
  ON mu.cod_ibge = m.cod_municipio
WHERE mu.nome_municipio ilike 'SANTA MARIA'
  AND mu.cod_ibge LIKE '43%' -- 43 = RS no codigo IBGE
  AND m.nu_ano_censo = 2022;
```

A consulta aproveita o fato de que a tabela de matrículas já está agregada por município e faixa etária, de modo que basta filtrar o município desejado, restringir a UF pelo prefixo do código IBGE e selecionar o ano de referência. Sem acesso a esse esquema, um LLM genérico pode até descrever, em termos qualitativos, que "Santa Maria é um polo universitário, com milhares de matrículas na faixa de 18 a 24 anos", mas dificilmente apresentará a expressão SQL correta (com as junções e colunas adequadas), tampouco o valor numérico exato.

2.1.3 Exemplo 3: evolução temporal de cursos na área de Computação

Por fim, considere uma consulta que envolve recorte por área de avaliação, região geográfica e série temporal:

> *"Entre 2013 e 2022, como evoluiu o número de cursos de pós-graduação na área de Ciência da Computação na região Sul?"*

O esquema inclui a tabela `capes_cursos_bruto`, que reúne informações sobre cursos de pós-graduação, com colunas como `an_base` (ano de referência), `nm_area_avaliacao` (por exemplo, "CIÊNCIA DA COMPUTAÇÃO"), `nm_regiao` (como "Sul") e `cd_curso_ppg` (identificador do curso). Uma forma razoável de expressar a consulta em SQL é:

```
SELECT
  c.an_base,
  COUNT(DISTINCT c.cd_curso_ppg) AS total_cursos_computacao_sul
FROM capes_cursos_bruto AS c
WHERE UPPER(c.nm_area_avaliacao) = 'CIÊNCIA DA COMPUTAÇÃO'
  AND c.nm_regiao = 'SUL'
  AND c.an_base BETWEEN 2013 AND 2022
GROUP BY c.an_base
ORDER BY c.an_base;
```

Aqui, o modelo precisa compreender que a área de interesse está codificada textualmente em `nm_area_avaliacao`, que a dimensão geográfica relevante é a região (e não apenas a unidade federativa) e que a medida desejada é o número de cursos distintos por ano. A resposta ideal, portanto, não é um único valor escalar, mas uma série temporal a ser visualizada em forma de gráfico. Se o modelo não tiver acesso a esse esquema (ou se ele estiver mal representado), é comum que devolva uma resposta apenas descritiva, como "o número de cursos de Ciência da Computação na região Sul cresceu de forma consistente entre 2013 e 2022", sem qualquer SQL correspondente e sem apresentar os números reais.

Esses três exemplos ilustram de maneira concreta a lacuna entre uma resposta meramente textual, ainda que plausível, e a geração de SQL robusto e executável sobre um banco de dados real. Em todos os casos, o desempenho percebido pelo usuário final depende não apenas da capacidade linguística do modelo, mas da sua fidelidade ao esquema efetivo. No contexto deste trabalho, essa observação motiva o desenvolvimento de uma plataforma que permita comparar, de forma sistemática, diferentes modelos de linguagem em consultas em português sobre bases educacionais brasileiras, avaliando tanto a correção sintática e semântica do SQL quanto a utilidade prática da resposta obtida. Essas limitações dos modelos anteriores à era dos LLMs, somadas à dificuldade de generalização para esquemas amplos e heterogêneos, motivam a adoção de modelos de linguagem de grande porte e de estratégias de orquestração da inferência, discutidas na subseção seguinte (Seção 2.2).

2.2 LLMs, orquestração da inferência e estratégias para *text-to-SQL*

Na era dos LLMs, a literatura indica que o desempenho em *text-to-SQL* é determinado menos pela arquitetura interna do modelo e mais pela forma como a inferência é orquestrada: desenho do *prompt*, seleção e ordenação de exemplos, serialização do

esquema, pós-checagens sintáticas, execução segura e ciclos de *feedback* (Hong et al., 2024; Hong et al., 2025). Nesta subseção, são discutidas as principais estratégias propostas, com ênfase naquelas que inspiram diretamente o desenho da plataforma desenvolvida neste trabalho.

2.2.1 DAIL-SQL: contexto como variável de primeira ordem

O estudo de Gao et al. (2024) investiga de modo sistemático como diferentes escolhas de contexto influenciam a acurácia da geração de SQL. Em vez de propor uma nova arquitetura de modelo, o trabalho compara quatro eixos de variação: serialização de esquema, seleção de exemplos, desenho do *prompt* e pós-processamento com verificação.

No eixo de serialização de esquema, o banco relacional é convertido em texto legível para o LLM. As variações testadas incluem listar todas as tabelas com colunas, incluir tipos de dados, explicitar chaves primárias e estrangeiras e incorporar exemplos de junções frequentes. Os resultados mostram que descrições mais ricas do esquema elevam a taxa de *execution accuracy* em consultas que exigem múltiplas junções, pois reduzem ambiguidades de alinhamento entre entidades textuais e colunas.

Quanto à seleção de exemplos, o DAIL-SQL contrasta estratégias por similaridade semântica (busca densa por *embeddings*), por diversidade (cobrir padrões distintos de SQL) e por dificuldade (priorizar exemplos com agregações, aninhamentos e subconsultas). Na prática, a combinação de similaridade e diversidade mitiga o risco de *overfitting* ao *prompt* e melhora a capacidade de generalização. A ordenação dos exemplos também se mostra relevante: exemplos simples em primeiro lugar e consultas mais complexas em seguida favorecem composições graduais de raciocínio.

No que se refere ao design do *prompt*, o trabalho compara instruções sucintas com instruções estruturadas. Estas últimas incluem objetivo, passos esperados, formato de saída, restrições (por exemplo, evitar `SELECT *`) e lembretes para citar colunas com nome de tabela. Instruções mais explícitas reduzem erros de referência a colunas ambíguas e elevam a taxa de SQL válida.

No pós-processamento, o DAIL-SQL integra verificações de sintaxe, normalização de identificadores, tentativas de reparo com reconsulta ao modelo e imposição automática de cláusulas como `LIMIT`. Essa etapa reduz falhas de execução e evita consultas com *scans* desnecessários. O mérito do trabalho está em demonstrar empiricamente que a configuração do contexto é variável de primeira ordem, justificando o esforço de engenharia dedicado à representação textual do esquema, à curadoria de exemplos e às camadas de verificação.

2.2.2 SQLGenie: pipeline industrial com múltiplas inferências e controle

Ghosh, Jain & Yenigalla (2025) descrevem um sistema de produção voltado à confiabilidade e eficiência, no qual a robustez não deriva de uma única chamada ao modelo, mas da composição de etapas em um pipeline cuidadosamente projetado. O fluxo inclui ingestão e *onboarding* do esquema, com padronização de metadados e anotações semânticas; geração primária de SQL a partir da pergunta, da serialização do esquema e de um conjunto compacto de exemplos; verificações sintáticas e normalização, garantindo compatibilidade dialetal e corrigindo erros comuns antes da execução; execução segura, com políticas que impedem instruções de escrita e consultas de alto custo; pós-checagem

semântica, em que resultados inconsistentes acionam nova rodada de inferência com *feedback* orientado por dados; e, por fim, registro e rastreabilidade, com histórico de *prompt*, versão do modelo, latência e métricas de sucesso.

Esse tipo de abordagem é particularmente relevante para aplicações públicas, como dados do INEP, em que é necessário equilibrar desempenho, segurança e auditabilidade. A ideia de registrar sistematicamente consultas, erros e correções dialoga diretamente com a proposta deste trabalho de utilizar o sistema como um *live benchmark* contínuo.

2.2.3 SDE-SQL: sondagens intermediárias e exploração orientada à execução

Xie, Dai & Jiang (2025) propõem o conceito de *self-driven exploration*. Em vez de gerar a SQL final de uma vez, o sistema realiza sondagens intermediárias (*SQL probes*) que testam hipóteses sobre o esquema e sobre chaves de junção. O processo segue três fases principais: formulação de hipóteses (pequenas consultas exploratórias, como contagens e amostras); execução e coleta de sinais (distribuições de valores, presença de *nulls*, coerência entre colunas); e refinamento da consulta final com base nas evidências coletadas. Essa abordagem melhora a *execution accuracy*, sobretudo em cenários com múltiplas tabelas e junções não triviais, ao custo de maior número de chamadas ao modelo e execuções no banco, o que torna necessário estabelecer orçamentos e *timeouts*.

2.2.4 Síntese de abordagens e implicações práticas

A Tabela 1 sintetiza algumas das principais linhas de pesquisa e suas implicações para sistemas reais, incluindo o sistema proposto neste trabalho.

Tabela 1 – Síntese de abordagens em trabalhos correlatos e implicações práticas.

Linha / Trabalho	**Técnicas principais**	**Implicações para sistemas reais**
RAT-SQL (Wang et al., 2020)	*Encoder* com relações de esquema e atenção em grafo.	Melhora o *schema linking*, mas exige treinamento especializado e dados anotados.
PICARD (Scholak; Schucher; Bahdanau, 2021)	Decodificação constrangida por gramática SQL.	Aumenta a proporção de SQL válida e reduz falhas de execução, à custa de maior complexidade na geração.
DAIL-SQL (Gao et al., 2024)	Avaliação sistemática de serialização de esquema, exemplos e *prompt*.	Mostra que contexto bem curado é determinante para acurácia e custo de inferência.
SQLGenie (Ghosh; Jain; Yenigalla, 2025)	Pipeline com múltiplas inferências, verificações e *feedback* orientado a dados.	Aumenta robustez, auditabilidade e eficiência em cenários de produção.
SDE-SQL (Xie; Dai; Jiang, 2025)	Sondagens SQL intermediárias (*probes*) e refinamento iterativo.	Reduz ambiguidades de junção; exige controle rigoroso de orçamento e latência.

Fonte: Elaborado pelo autor a partir da literatura citada.

A partir dessas referências, a plataforma proposta neste trabalho adota explicitamente o princípio de que a orquestração é parte central do projeto: a serialização de esquema é informativa, os exemplos são selecionados de forma híbrida, há camadas de pós-checagem e execução segura, e o sistema registra métricas de latência e custo para fins de análise comparativa.

2.2.5 Estratégias de *prompt engineering*, pré-processamento e pós-checagem

Complementando as abordagens anteriores, a literatura destaca um conjunto de estratégias transversais que também são incorporadas neste trabalho. Em termos de *prompt engineering*, recomenda-se explicitar objetivo, passos esperados, formato de resposta e restrições (por exemplo, evitar `SELECT *`, citar colunas com o nome da tabela e usar *alias* consistentes) (Hong et al., 2025; Gao et al., 2024). A seleção de exemplos *few-shot* normalmente combina recuperação por *embeddings* da pergunta com critérios de diversidade estrutural (padrões com `GROUP BY`, subconsultas, janelas, entre outros).

No pré-processamento de esquema, destacam-se práticas como normalização de nomes (remoção de abreviações opacas, padronização em _snake_case), construção de dicionários de sinonímia (mapeando termos do usuário para colunas específicas) e redução do esquema textual a subconjuntos relevantes para economizar *tokens* e diminuir confusão. Já no pós-processamento e na execução segura, são recorrentes técnicas como decodificação constrangida (PICARD), normalização de dialeto, reparo de consultas com base em mensagens de erro, imposição de cláusulas `LIMIT` e políticas que bloqueiam instruções de escrita, comandos DDL ou consultas de alto custo.

Em contextos públicos, custos e latência tornam-se restrições operacionais. Estratégias com múltiplas inferências (como no SQLGenie) e sondagens (como no SDE-SQL) elevam a robustez, mas exigem orçamentos bem definidos. A orquestração deve, portanto, equilibrar correção, custo e tempo de resposta. No domínio educacional, recomenda-se ainda que a explicabilidade não seja tratada como pós-texto opcional: traduzir a SQL executada para uma descrição natural e explicar tabelas, filtros e agregações aumenta a confiança e facilita a formação de usuários. Esse princípio orienta diretamente a arquitetura da plataforma desenvolvida neste TCC.

2.3 Avaliação, *benchmarks* e limites das métricas tradicionais

Os *benchmarks* de *text-to-SQL* desempenham papel central no avanço da área, servindo como referência para avaliação de desempenho, generalização e robustez das abordagens. Entre os mais influentes estão o *Spider*, o *SParC* e o *CoSQL* (Yu et al., 2018; Yu et al., 2019; Lab; Salesforce, 2019), que estabeleceram os primeiros padrões amplamente aceitos para a tarefa.

O *Spider* introduziu o conceito de *cross-domain generalization*, em que o modelo precisa lidar com esquemas de bancos não vistos durante o treinamento. Esse desenho foi essencial para afastar o viés de memorização, mas traz limitações significativas: as perguntas são formuladas em inglês, seguem padrões linguísticos controlados e dependem de anotações humanas relativamente homogêneas, o que reduz a diversidade real do corpus. Em seguida, o *SParC* e o *CoSQL* estenderam o *Spider* para cenários de múltiplas rodadas (*multi-turn*), simulando conversas em que o usuário refina progressivamente a consulta.

Apesar de mais realistas, essas extensões mantêm um escopo controlado, vocabulário restrito e pouca diversidade de domínios linguísticos.

O lançamento do *BIRD* (Team, 2023) representou um avanço ao introduzir a métrica de *valid efficiency score*, que considera não apenas a correção semântica da SQL, mas também sua eficiência de execução. Isso aproxima a avaliação do uso prático, no qual consultas sintaticamente válidas, mas ineficientes, são penalizadas. Ainda assim, o *BIRD* permanece centrado em dados estáticos, de domínio empresarial e predominantemente em inglês. Estudos recentes, como Zhang (2024), discutem o impacto de ruído, redundância e contaminação nesses conjuntos, sugerindo que *benchmarks* tradicionais não capturam bem o comportamento dos modelos em domínios reais com perguntas livres e esquemas amplos.

Em resposta a essas limitações, surgem propostas como o *LiveSQLBench* (Team, 2024), que adota a ideia de um *benchmark* evolutivo e livre de contaminação, no qual os esquemas mudam ao longo do tempo e as consultas são geradas por usuários reais. A versão pública inicial (*Base Lite*) reforça a tendência de aproximar avaliação e uso prático, incorporando métricas relacionadas a custo, eficiência e julgamento humano.

A Tabela 2 resume as características, métricas e limitações dos principais *benchmarks*.

Tabela 2 – Comparativo entre principais *benchmarks* de *text-to-SQL*.

Benchmark	**Características principais**	**Métricas utilizadas**	**Limitações identificadas**
Spider (2018)	Consultas complexas e múltiplos bancos; primeira avaliação *cross-domain*.	*Exact Match*, *Execution Accuracy*.	Dados estáticos, domínio limitado e idioma único (inglês).
SParC (2019)	Consultas contextuais em múltiplas rodadas (*multi-turn*).	*Contextual EM*, *Turn-Level Accuracy*.	Escopo controlado e forte dependência de anotadores humanos.
CoSQL (2019)	Conversas interativas simuladas entre usuário e sistema.	*Dialogue Accuracy*, *Turn EM*.	Baseia-se em padrões conversacionais artificiais e limitados.
BIRD (2023)	Introduz avaliação de eficiência e custo de execução.	*Valid Efficiency Score*, *Runtime*.	Domínio empresarial e ausência de diversidade linguística.
LiveSQLBench (2024)	*Benchmark* evolutivo, orientado a consultas reais, com foco em contaminação reduzida.	*Execution Accuracy*, *Cost Efficiency*, *Human Evaluation*.	Em expansão contínua; versões completas ainda em desenvolvimento e métricas em fase de padronização.

Fonte: Elaborado pelo autor a partir de Yu et al. (2018), Yu et al. (2019), Team (2023), Zhang (2024), Team (2024).

Essas comparações evidenciam uma lacuna relevante para este TCC: a maioria dos *benchmarks* avalia apenas consultas pré-definidas e ignora a interação humana em tempo real. Em contextos aplicados, como o educacional, essa limitação torna métricas clássicas (por exemplo, *exact match*) insuficientes para capturar utilidade prática, clareza e confiança do usuário. O presente trabalho busca superar essa restrição ao adotar uma plataforma dinâmica e interativa, na qual a avaliação é feita em tempo real e as consultas partem de perguntas espontâneas de usuários humanos, complementando métricas automáticas com julgamento humano sobre correção, utilidade e clareza da explicação.

2.4 Ferramentas relacionadas de observabilidade e avaliação

Embora o *QueryLab* esteja focado em consultas a bases relacionais, sua concepção dialoga com um conjunto de ferramentas que surgiram para observabilidade, avaliação e depuração de interações com modelos de linguagem de forma geral. Entre elas, destacam-se:

- PromptLayer, que registra histórico de *prompts*, respostas e versões de modelos, permitindo rastreabilidade e reprodutibilidade de experimentos;

- Mirascope e Lilypad, que fornecem abstrações para padronizar, registrar e depurar chamadas a modelos de linguagem em aplicações variadas;
- LangSmith, que integra com a biblioteca LangChain e oferece recursos de avaliação, conjuntos de testes e *tracing* detalhado de pipelines;
- Weights & Biases Weave, que estende a plataforma de experimentos da Weights & Biases para observabilidade de fluxos com LLMs.

Essas ferramentas apontam para a importância de tratar interações com modelos de linguagem como objetos de observação contínua, com métricas, logs e histórico estruturado. O *QueryLab* se aproxima dessa visão ao registrar, de forma sistemática, as hipóteses geradas por diferentes modelos, seus resultados e as avaliações fornecidas pelos usuários, o que abre caminho para futuras integrações com plataformas de observabilidade mais amplas.

2.5 Trabalhos correlatos na UFSM, lacunas e posicionamento deste trabalho

No contexto da Universidade Federal de Santa Maria (UFSM), a aplicação de inteligência artificial para consultas em linguagem natural sobre dados educacionais públicos tem sido objeto de pesquisa crescente, especialmente nos cursos de Engenharia de Computação e Ciência da Computação. Essa linha consolidou-se a partir do uso de dados do Instituto Nacional de Estudos e Pesquisas Educacionais Anísio Teixeira (INEP), com foco inicial no Censo da Educação Superior.

O estudo de Trevisan (2024) teve como principal objetivo desenvolver um protótipo de tradução automática de perguntas em português para comandos SQL. O trabalho utilizou aprendizado supervisionado tradicional, com base em pares de treinamento compostos por perguntas e consultas correspondentes, e foi aplicado sobre um subconjunto do Censo da Educação Superior. A avaliação concentrou-se em métricas de precisão sintática e semântica das consultas, bem como na adequação das respostas retornadas. O projeto demonstrou a viabilidade técnica do uso de redes neurais para o problema, mas apresentou limitações de escalabilidade, pois dependia de um conjunto estático de perguntas e não oferecia interface de interação com o usuário final.

Em continuidade, Assumpção (2025) realizaram uma análise comparativa entre quatro modelos de linguagem abertos — XiYan-SQL, OneSQL, OmniSQL e SQLCoder — aplicando-os sobre o mesmo domínio de dados do INEP. O estudo utilizou métricas de *execution accuracy*, *exact match* e *valid efficiency score*, e analisou o custo de execução e a coerência das consultas geradas. Entre os resultados, observou-se que modelos treinados com raciocínio simbólico explícito apresentaram melhor desempenho em consultas com múltiplas junções. Contudo, o estudo manteve uma abordagem experimental fechada, sem considerar interações em tempo real, variação de idioma ou explicabilidade das respostas.

A Tabela 3 apresenta uma comparação detalhada entre os dois estudos conduzidos na UFSM e o presente projeto.

Tabela 3 – Comparativo detalhado entre os estudos da UFSM e o presente projeto.

Aspecto	**Trevisan (2024)**	**Assumpção (2025)**	**Este trabalho (2025)**
Domínio	Censo da Educação Superior (INEP)	Censo da Educação Superior (INEP)	Censo da Educação Superior (INEP)
Abordagem principal	Aprendizado supervisionado	Modelos LLM *open-source*	LLMs multimodelo e dinâmicos
Base de avaliação	Conjunto fixo de perguntas	Conjunto fixo de perguntas	Consultas livres e interativas
Métricas	Correção sintática e semântica	*Execution Accuracy*, *Exact Match*, *Efficiency Score*	Métricas automáticas e avaliação humana
Saída gerada	SQL puro	SQL validado	SQL e resposta em linguagem natural explicada
Interface	Notebook em Python	Scripts comparativos	Plataforma web interativa
Contribuição	Prova de conceito inicial	Comparação de modelos abertos	Plataforma dinâmica, explicável e comparativa

Fonte: Elaborado pelo autor.

Esses trabalhos constituem um alicerce relevante para a pesquisa atual, pois exploraram tanto a viabilidade técnica da tradução de consultas quanto a comparação entre diferentes arquiteturas. No entanto, observam-se lacunas importantes, sintetizadas na Tabela 4.

Tabela 4 – Principais lacunas e oportunidades identificadas nos trabalhos da UFSM.

Lacuna identificada	**Oportunidade de avanço explorada neste trabalho**
Falta de interação em tempo real com o usuário	Desenvolvimento de uma interface web que permite consultas livres e feedback imediato.
Ausência de geração de respostas explicáveis	Implementação de módulo de explicação textual da SQL executada e de seus resultados.
Dependência de conjuntos fixos de perguntas	Criação de ambiente dinâmico e aberto, permitindo uso como *live benchmark*.
Foco restrito a métricas automáticas	Inclusão de avaliação humana baseada em clareza, correção e utilidade.
Falta de telemetria e controle de latência	Integração de medição automática de tempo de inferência e custo de *tokens*.

Fonte: Elaborado pelo autor.

Em síntese, o presente trabalho posiciona-se como um desdobramento natural das iniciativas da UFSM, incorporando elementos de interatividade, comparabilidade e explicabilidade que os estudos anteriores não contemplavam. Ao adotar uma arquitetura orientada por múltiplos LLMs, telemetria em tempo real e políticas de execução segura, esta pesquisa avança o estado da arte institucional e responde diretamente às lacunas

identificadas: oferece uma plataforma que funciona simultaneamente como ferramenta de consulta, ambiente de avaliação contínua e *live benchmark* para modelos de *text-to-SQL* no domínio educacional brasileiro.

3 METODOLOGIA, PROJETO E IMPLEMENTAÇÃO

Esta seção descreve a metodologia adotada no desenvolvimento do *QueryLab*, detalhando as decisões de projeto, a arquitetura do sistema, o fluxo de orquestração entre modelos de linguagem e banco de dados, bem como o processo de avaliação com usuários. O objetivo é explicitar, desde o início, quais funcionalidades a plataforma oferece, como elas foram implementadas e de que forma permitem comparar, em condições reais de uso, diferentes modelos de Inteligência Artificial capazes de traduzir perguntas em linguagem natural para consultas SQL sobre um banco de dados relacional.

O desenvolvimento seguiu uma abordagem de prototipação evolutiva, com ciclos iterativos de implementação, experimentação e refinamento. Inicialmente foi construída uma versão mínima viável que ligava um único modelo de linguagem ao banco de dados. A partir da análise de suas limitações, a plataforma foi estendida para suportar múltiplos modelos e, posteriormente, para executar esses modelos em paralelo, permitindo comparação direta entre as hipóteses geradas para uma mesma pergunta. Em paralelo, foram introduzidos mecanismos de telemetria, registro de experimentos e interface para avaliação humana.

3.1 Metodologia geral e requisitos da plataforma

De forma sintética, a metodologia pode ser organizada em quatro eixos que nortearam o desenvolvimento: (i) modelagem do problema; (ii) projeto da arquitetura; (iii) orquestração multi-LLM; e (iv) avaliação com usuários. Em vez de tratar esses elementos apenas como uma sequência de tarefas, eles funcionam aqui como um fio condutor que conecta os objetivos teóricos da pesquisa à implementação concreta do sistema.

Na modelagem do problema, foi definido o escopo de atuação do *QueryLab* a partir da base de dados educacionais do INEP, já utilizada em trabalhos anteriores da UFSM. Foram identificados os tipos de perguntas que o sistema deveria suportar (por exemplo, contagens, comparações entre regiões, análises por tipo de administração) e os principais desafios de mapeamento entre a linguagem natural e o esquema relacional.

No projeto da arquitetura, optou-se por uma estrutura em camadas que separa claramente a interface de usuário, a lógica de orquestração entre modelos, a execução segura de SQL e a persistência de resultados e métricas. Essa separação facilita a manutenção, permite a substituição ou inclusão de novos modelos de linguagem e reduz o acoplamento entre componentes.

A orquestração multi-LLM foi concebida para receber a pergunta do usuário, preparar o contexto de esquema, disparar chamadas para diferentes modelos de linguagem em paralelo e consolidar as respostas em uma interface de comparação. A lógica de orquestração incorpora práticas da literatura, como pós-checagem de consultas, execução segura e registro detalhado de cada hipótese gerada.

Por fim, a avaliação com usuários foi planejada para ocorrer em um contexto real de uso, envolvendo estudantes e profissionais da UFSM. A plataforma registra métricas automáticas (taxa de sucesso de execução, tempo de resposta, número de linhas retornadas,

entre outras), enquanto um formulário complementar coleta percepções qualitativas, como clareza, utilidade e modelo preferido.

A partir desses eixos, foram derivados os principais requisitos funcionais da plataforma, que podem ser entendidos como um pequeno *backlog* de alto nível:

- permitir que o usuário formule perguntas em linguagem natural, em português, sobre a base do INEP;
- gerar, para cada pergunta, uma ou mais consultas SQL por meio de diferentes modelos de linguagem;
- executar as consultas de forma segura em um banco de dados real, com limitação de recursos e bloqueio de comandos destrutivos;
- apresentar ao usuário, lado a lado, as hipóteses geradas por cada modelo, incluindo SQL, explicação textual e resultados;
- registrar métricas de latência, taxa de sucesso de execução, número de linhas retornadas e custos de inferência;
- permitir que o usuário avalie a qualidade das respostas em diferentes dimensões;
- armazenar o histórico de sessões, perguntas, respostas e avaliações para análise posterior.

Os tópicos seguintes detalham como esses requisitos foram concretizados por meio da arquitetura proposta, do fluxo de orquestração e das implementações de frontend e backend.

3.2 Visão geral do sistema e arquitetura em camadas

Do ponto de vista do usuário, o *QueryLab* funciona como uma interface web em que se escreve uma pergunta em português e se obtém, em resposta, uma explicação em linguagem natural e o comando SQL correspondente. Internamente, essa interação envolve múltiplos estágios de processamento, que vão desde a classificação da intenção até a execução segura da consulta no banco de dados e a coleta de métricas.

A Figura 1 apresenta a visão global da arquitetura do sistema, organizada em camadas lógicas que separam preocupações de apresentação, orquestração, inferência e persistência.

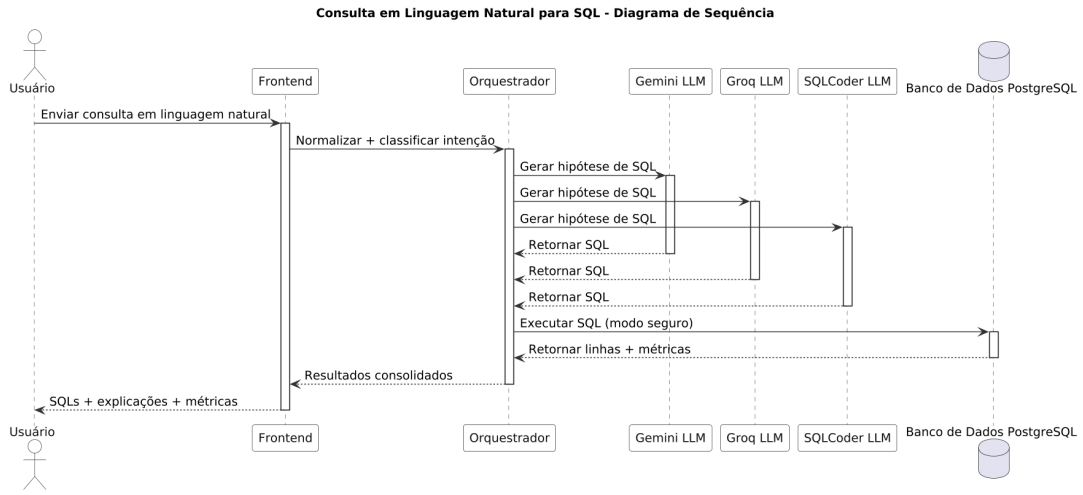

Figura 1 – Diagrama de sequência do *QueryLab*, com separação entre frontend, backend, camada de orquestração e bancos de dados.

A Tabela 5 resume os principais componentes dessa arquitetura, destacando sua função e relação com a literatura revisada na Seção 2.

Tabela 5 – Componentes principais da arquitetura do sistema *QueryLab*.

Camada	**Função principal**	**Relação com literatura e decisões de projeto**
Frontend	Interface de interação, histórico e avaliação.	Adota explicabilidade e transparência conforme diretrizes do SQLGenie e princípios de *XAI*.
Backend	Orquestra requisições, aplica regras de negócio e valida SQL.	Implementa *multi-model routing* e *fallbacks*, inspirados na abordagem de orquestração do DAIL-SQL.
Orquestrador LLM	Gera consultas em múltiplos modelos a partir de uma mesma pergunta e controla *prompts* e pós-checagem.	Baseado em estratégias de controle de contexto e verificação contínua (*post-checking*) conforme Gao et al. (2024) e Scholak, Schucher & Bahdanau (2021).
Banco de Dados INEP	Base de dados relacional utilizada nas consultas.	Permite replicar cenários de consultas reais sobre dados públicos, aproximando-se da proposta de *live benchmarks*.
Banco Operacional	Registro de sessões, histórico, avaliações e métricas.	Garante rastreabilidade e telemetria, conforme práticas recomendadas por Ghosh, Jain & Yenigalla (2025).

Fonte: Elaborado pelo autor.

O núcleo do sistema, implementado em Node.js, atua como mediador entre os LLMs e o PostgreSQL. O módulo de orquestração permite alternância entre modelos, aplicando políticas de validação (*post-checking*) e execução segura (*safe-exec*). A camada de inferência inclui ainda o módulo de *telemetria*, responsável por medir latência e custo

de tokens em cada modelo, o que possibilita comparações diretas entre provedores, em linha com a preocupação com eficiência operacional introduzida pelo *BIRD* (Team, 2023).

Essa organização em camadas permite que a plataforma seja estendida com novos modelos de linguagem ou novas bases de dados sem interferir na interface ou na lógica principal de negócios, reforçando o caráter de laboratório de experimentação do *QueryLab*.

3.3 Fluxo de orquestração entre usuário, modelos e banco de dados

A partir da arquitetura geral, o fluxo básico de orquestração pode ser descrito passo a passo. O *QueryLab* atua como um intermediário entre usuários e a base relacional: recebe perguntas em linguagem natural, classifica a intenção, prepara o contexto de esquema, aciona múltiplos modelos de linguagem, valida e executa as consultas geradas, registra telemetria e devolve resultados explicados.

Esse fluxo é representado pelo Algoritmo 1, que descreve a sequência lógica de etapas que o backend executa ao receber uma pergunta em linguagem natural, contemplando autenticação, determinação de intenção, preparação de contexto, geração de SQL, validação, execução e registro de métricas.

Algorithm 1: Orquestração do *QueryLab* para perguntas em linguagem natural

Entrada: Pergunta do usuário q (PT-BR); metadados do usuário; conexão com BD relacional; catálogo de esquemas

Saída: Resposta em linguagem natural, trecho SQL e métricas de execução

1. Autenticação e contexto
Recuperar *tenant*, permissões e preferências do usuário
Carregar instruções e exemplos do domínio, se houver
2. Classificação de intenção
$intent \leftarrow$ ClassificarIntencao$(q) \in \{\text{CONVERSA}, \text{QUERY}, \text{SCHEMA}\}$
if $intent$ = *CONVERSA* **then**
  Responder via modelo conversacional (sem SQL) e **retorna**
if $intent$ = *SCHEMA* **then**
  Recuperar e devolver esquemas solicitados (tabelas, colunas, chaves)
  retorna
3. Preparação de contexto para SQL
$schema \leftarrow$ SelecionarEsquema(q, catálogo)
$constraints \leftarrow$ políticas de segurança (filtros de linha, limites de tempo e de linhas)
$hints \leftarrow$ exemplos de consultas equivalentes e vocabulário do domínio
4. Geração de SQL
$draftSQLs \leftarrow$ GerarSQLMultiModelo$(q, schema, hints)$ `// método paralelo`
5. Validação e execução segura
foreach sql *em* $draftSQLs$ **do**
  $sqlNorm \leftarrow$ NormalizarSQL(sql)
  $result \leftarrow$ ValidarEExecutar$(sqlNorm, schema, constraints)$
  Registrar métricas e status de execução para cada hipótese
6. Pós-processamento
Construir respostas em linguagem natural para cada hipótese bem-sucedida
Anexar trechos de SQL, métricas de tempo e contagem de linhas
7. Telemetria e auditoria
Registrar {hash de q, modelos utilizados, esquema, métricas e usuário} sem dados sensíveis
8. Entrega
Devolver conjunto de hipóteses, permitindo comparação lado a lado e seleção pelo usuário

A resposta final inclui não apenas o resultado tabular, mas também um resumo interpretativo e indicadores como latência, tokens processados e tempo de execução. Esse retorno espelha as boas práticas de explicabilidade observadas em sistemas como o SQLGenie (Ghosh; Jain; Yenigalla, 2025) e o SDE-SQL (Xie; Dai; Jiang, 2025), em que o usuário compreende não apenas a resposta, mas também o processo que a gerou.

3.4 Evolução arquitetural e método paralelo

O desenvolvimento do sistema ocorreu de forma incremental, estruturado em três fases principais. Na primeira etapa, implementou-se um pipeline sequencial em que apenas um modelo de linguagem era responsável por gerar SQL a partir da pergunta do usuário. Em uma segunda fase, novos modelos foram incorporados de forma experimental, permitindo alternância manual entre provedores. Na fase atual, o sistema executa três modelos em paralelo, consolidando as hipóteses produzidas em uma única visão comparativa para o usuário.

O método paralelo consiste em enviar a mesma pergunta, com o mesmo contexto de esquema e exemplos, simultaneamente para Gemini, Groq e SQLCoder. O backend aguarda as três respostas ou até um tempo máximo configurado. Em seguida, cada hipótese é validada, executada e registrada com suas métricas específicas. Do ponto de vista do usuário, a consulta é feita uma única vez e os resultados aparecem lado a lado, como ilustrado na Figura 2.

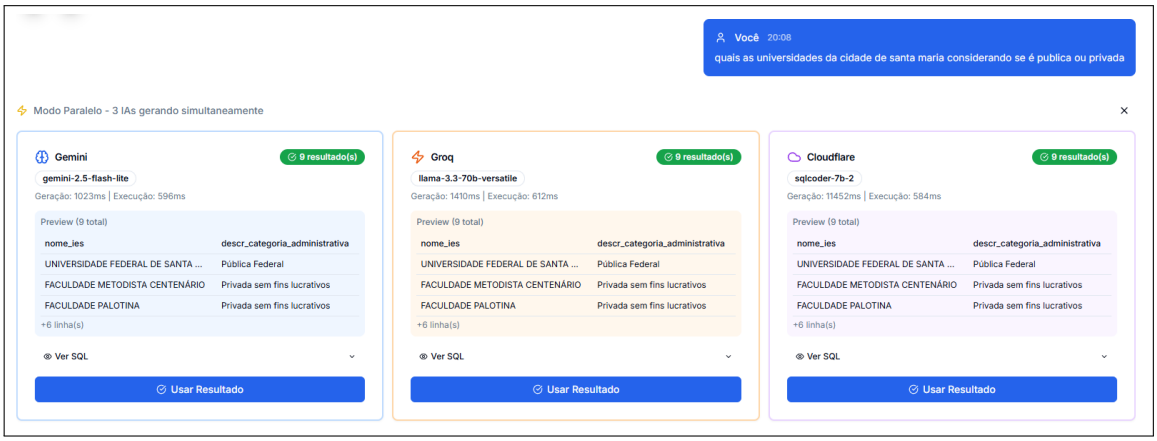

Figura 2 – Interface de comparação paralela entre Gemini, Groq e SQLCoder para uma mesma pergunta.

Esse método trouxe ganhos significativos para o projeto. Em vez de submeter várias vezes a mesma pergunta para modelos diferentes, a comparação passou a fazer parte do fluxo padrão de uso, o que facilita a análise de divergências, a identificação de erros e a escolha da resposta mais alinhada à intenção do usuário. Além disso, a presença de múltiplas hipóteses permite que o usuário selecione não apenas a consulta mais correta, mas também aquela que considere mais interessante, por ser mais detalhada, mais simples ou até mesmo inesperada.

3.5 Implementação do frontend

O frontend do *QueryLab* foi implementado em React, com foco em acessibilidade (WCAG 2.1), responsividade, clareza visual e facilidade de comparação entre consultas. A interface foi desenhada para que o usuário possa formular perguntas em linguagem natural sem precisar conhecer SQL, reduzindo a carga cognitiva de uso. A arquitetura de componentes reutilizáveis também simplifica a manutenção e permite a evolução incremental da ferramenta.

Do ponto de vista da experiência de uso, o fluxo principal da interface pode ser dividido em quatro momentos: (i) a tela inicial de boas-vindas e configuração; (ii) a comparação paralela entre modelos; (iii) a exploração da resposta selecionada, em formato de chat e visão técnica; e (iv) a etapa de avaliação qualitativa das hipóteses geradas.

Na **tela inicial**, o usuário encontra uma interface de boas-vindas com textos e cards explicativos sobre a plataforma, seleção de modelos e do contexto de dados, informações sobre o estado de conexão e um histórico recente de sessões. Essa organização busca contextualizar rapidamente o propósito do sistema e oferecer exemplos de uso, ajudando o usuário a elaborar suas primeiras perguntas. A Figura 3 ilustra essa tela.

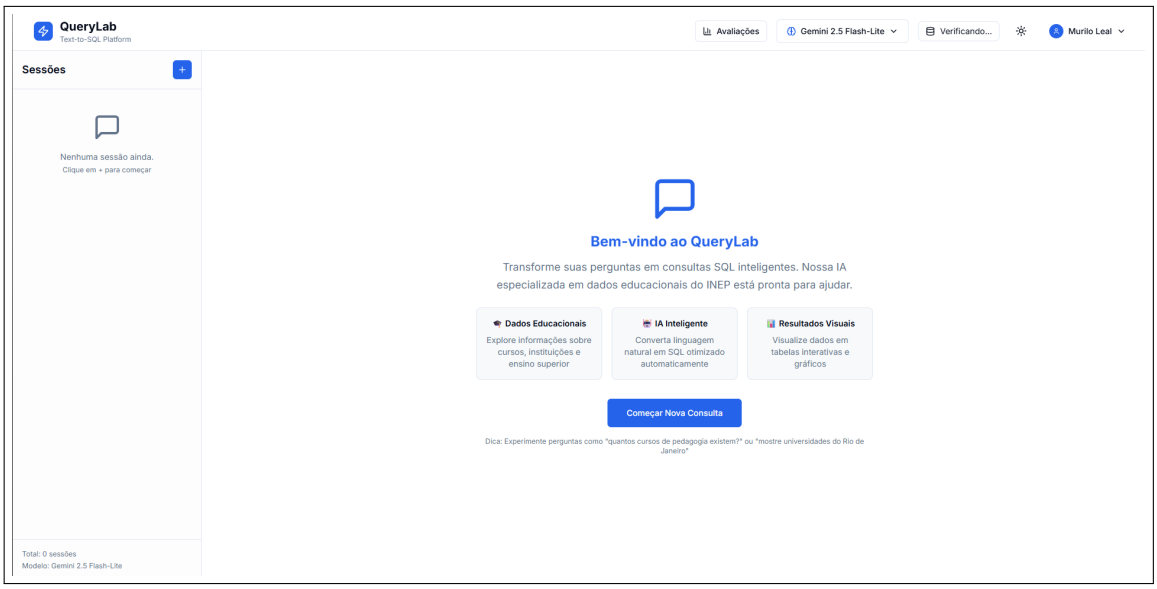

Figura 3 – Tela inicial do *QueryLab*, com mensagens e cards explicativos sobre o produto, seleção de modelos, tema, informações de conexão e histórico de sessões.

Após o envio da pergunta, o frontend passa para a **visão de comparação**, em que as respostas dos modelos são apresentadas lado a lado, em colunas, uma para cada modelo configurado. Para cada coluna, a interface exibe o comando SQL gerado, uma explicação textual em linguagem natural e as principais métricas de execução (como tempo e número de linhas retornadas), conforme ilustrado previamente na Figura 2. Essa disposição paralela foi pensada para evidenciar divergências entre as hipóteses e apoiar o usuário na escolha da resposta mais adequada ao seu objetivo.

Ao selecionar uma das hipóteses, o usuário é levado à etapa de **exploração da resposta**. Nessa etapa, a interface oferece duas visões complementares: uma visão técnica e uma visão em formato de chat. A **visão técnica**, mostrada na Figura 4, destaca a SQL gerada, o tempo de execução, a quantidade de linhas retornadas e uma amostra dos dados obtidos. Essa tela é voltada a usuários que desejam inspecionar o resultado em maior detalhe ou reutilizar a consulta em outros contextos.

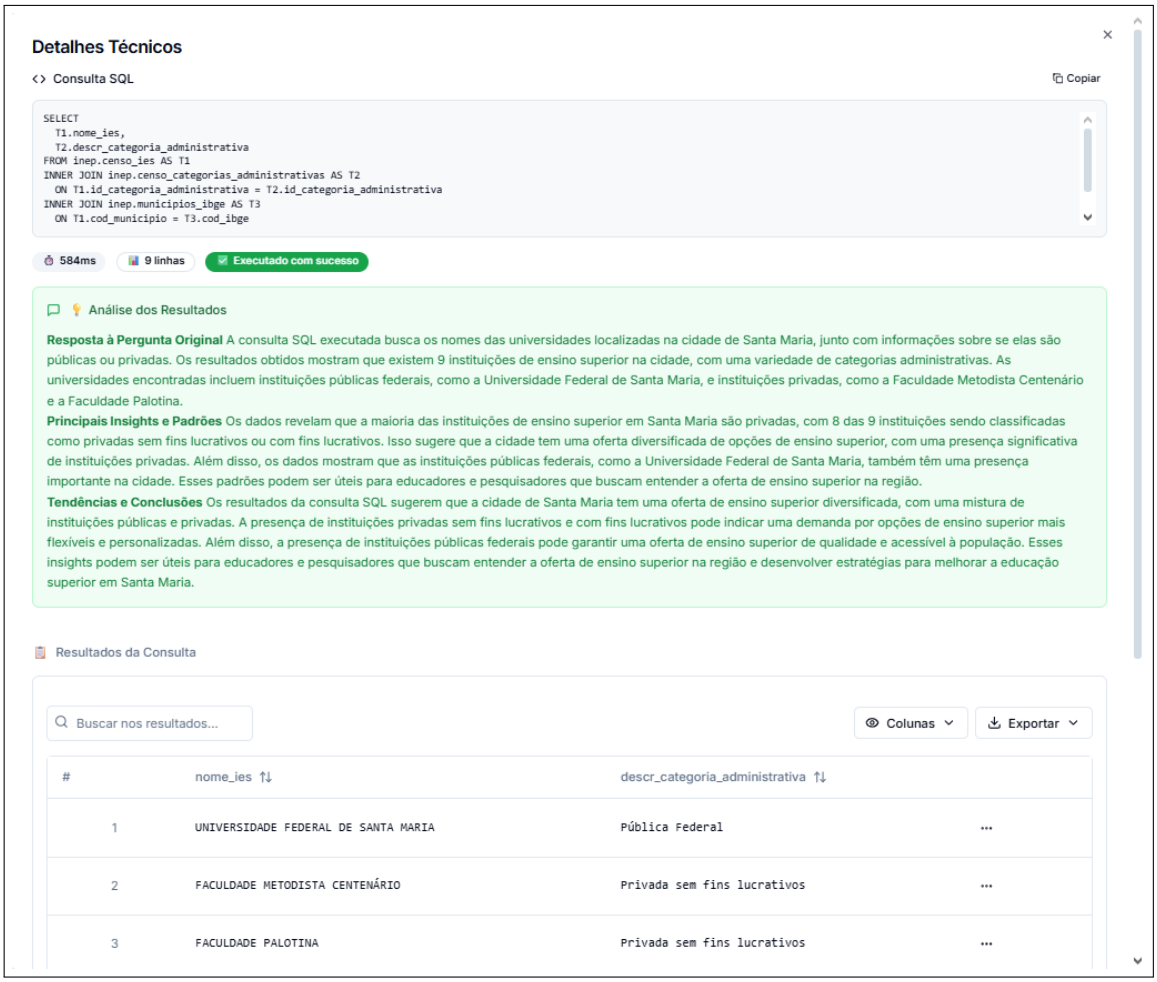

Figura 4 – Visão técnica da execução, com SQL gerada, tempo de execução, linhas retornadas e amostra de dados.

Complementarmente, a **visão em formato de chat**, ilustrada na Figura 5, apresenta a resposta selecionada em linguagem natural, em um estilo conversacional mais próximo de assistentes virtuais. Nessa tela, o usuário visualiza o enunciado da pergunta, a explicação produzida pelo modelo e eventuais orientações adicionais sobre como interpretar os resultados. Essa abordagem favorece a compreensão por parte de usuários menos familiarizados com SQL ou com termos técnicos de bancos de dados.

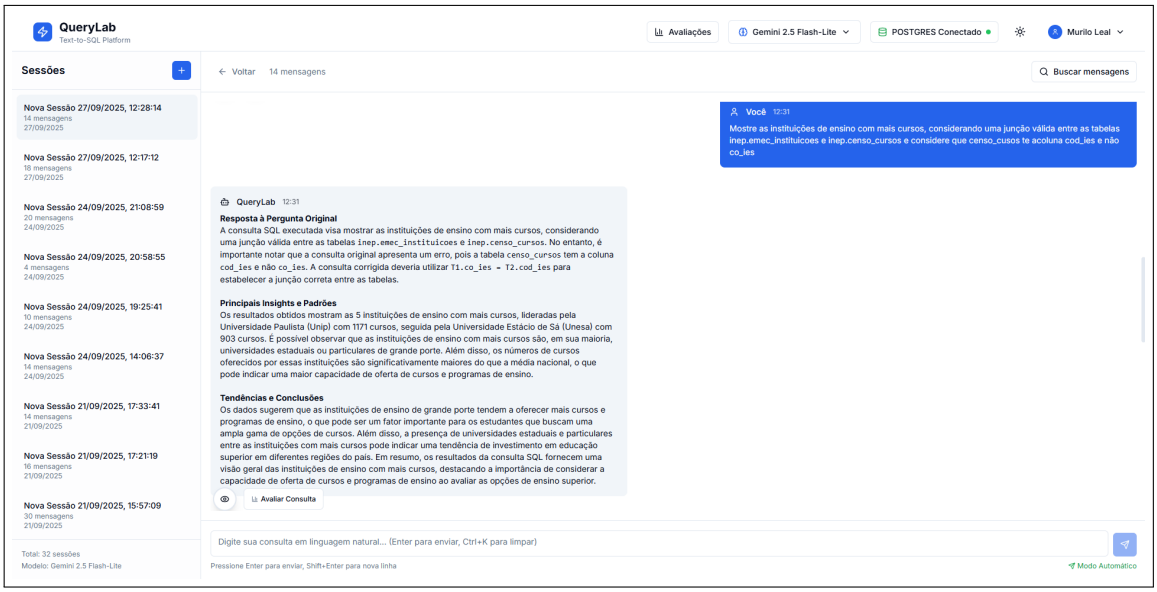

Figura 5 – Visão em formato de chat, com a resposta em linguagem natural para a hipótese selecionada.

Por fim, o frontend contempla uma etapa específica de **avaliação**, em que o usuário pode atribuir notas a cada hipótese de SQL gerada. A interface permite avaliar diferentes dimensões de qualidade, como correção, precisão, performance, completude e clareza da explicação. Esse processo complementa as métricas automáticas do sistema e contribui para a análise comparativa entre os modelos, incorporando uma perspectiva humana à avaliação. A Figura 6 apresenta a tela dedicada a essa etapa.

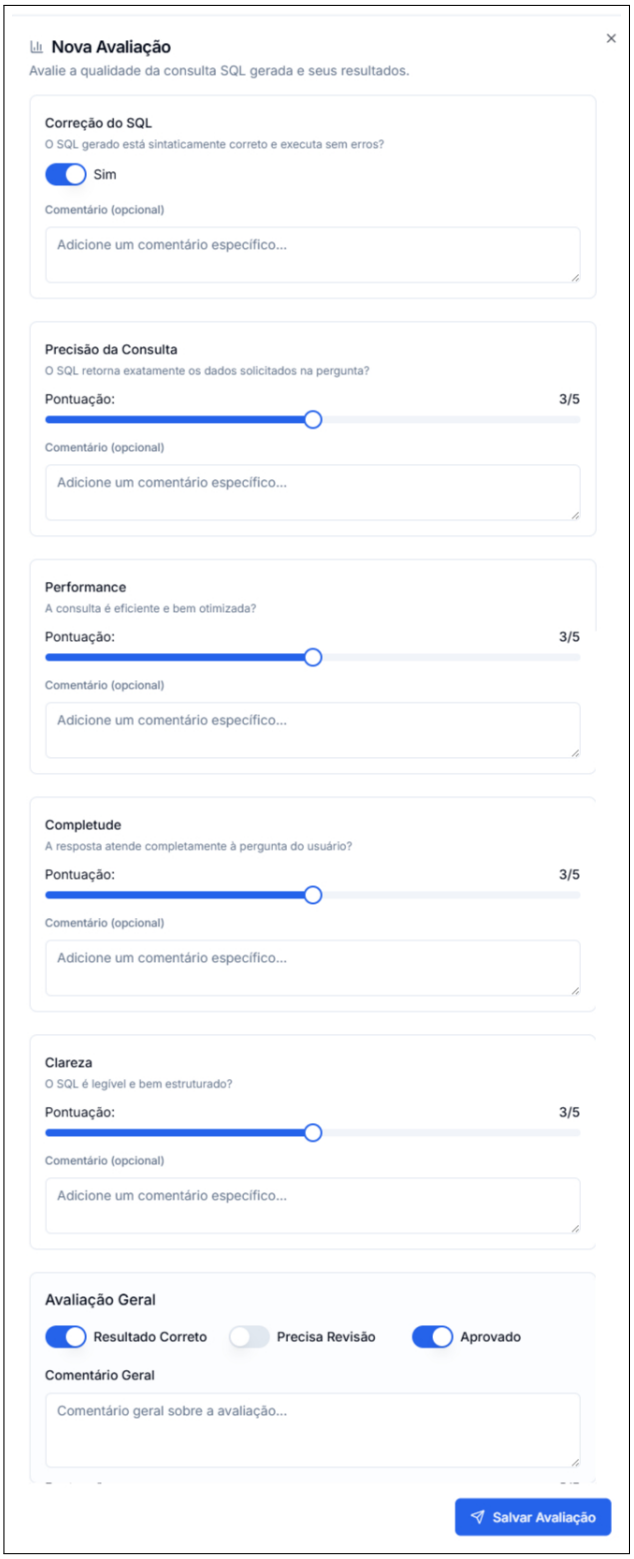

Figura 6 – Tela de avaliação da qualidade das consultas SQL geradas, com notas atribuídas pelos usuários em diferentes dimensões.

3.6 Backend, segurança e validação das consultas SQL

O backend foi implementado em Node.js com Express e é responsável por orquestrar todo o fluxo entre a interface, os modelos de linguagem e o banco de dados. Ele centraliza regras de negócio, validações e políticas de segurança, buscando evitar riscos como injeção de SQL ou acessos indevidos à base.

A segurança é tratada em várias camadas: autenticação e autorização por token, isolamento entre o banco de dados público de análise e o banco operacional, validação de comandos SQL e limitação de recursos. Consultas destrutivas, como `DROP`, `DELETE`, `ALTER` e instruções de modificação de dados, são bloqueadas na etapa de validação, e o sistema impõe limites de tempo e de número de linhas retornadas para evitar uso abusivo ou acidental.

Uma vez geradas as hipóteses de SQL pelos modelos, o backend aplica uma série de verificações antes da execução no PostgreSQL. A Tabela 6 resume as principais regras de validação.

Tabela 6 – Regras de validação aplicadas às consultas geradas pelos modelos.

Regra	**Descrição**
Detecção de comandos proibidos	Bloqueia comandos de definição ou modificação de estrutura e conteúdo, como `DROP`, `DELETE`, `ALTER`, `TRUNCATE`.
Verificação de múltiplas instruções	Impede uso indevido de `;` que permita agrupar várias consultas em uma mesma requisição.
Limitação de recursos	Impõe limites máximos de tempo de execução e de número de linhas retornadas.
Análise sintática	Utiliza o próprio PostgreSQL para verificar se a consulta é sintaticamente válida antes de sua execução completa.
Registro de erros	Em caso de falha, registra o erro retornado pelo banco e o associa à hipótese e ao modelo correspondente.

Fonte: elaborado pelo autor.

Além dessas validações gerais, o sistema emprega mecanismos adicionais que inspecionam a estrutura textual e semântica das consultas. A etapa de verificação inicial garante que apenas instruções de leitura sejam aceitas: a query normalizada deve iniciar por `SELECT` ou `WITH`, e é automaticamente rejeitada caso contenha comandos de DML ou DDL como `INSERT`, `UPDATE`, `DELETE`, `DROP`, `CREATE`, `ALTER`, `TRUNCATE`, `GRANT`, `REVOKE` ou `EXEC/EXECUTE`. A validação também verifica a presença de palavras-chave frequentemente associadas a vetores avançados de injeção, como `xp_cmdshell`, `sp_configure`, `openrowset`, `opendatasource`, `bulk insert`, `load_file`, `into outfile` e `into dumpfile`.

Parte dessas checagens se apoia em expressões regulares que auxiliam na extração e análise da estrutura da consulta. Um dos padrões utilizados é:

```
/(?:from|join)\s+([a-zA-Z_][a-zA-Z0-9_.]*)/gi
```

Esse regex identifica referências a tabelas após as cláusulas `FROM` e `JOIN`, permitindo ao backend verificar se elas pertencem ao esquema autorizado. Outro conjunto de padrões identifica consultas agregadas contendo funções como `COUNT`, `SUM`, `AVG`, `MAX`,

MIN, bem como cláusulas GROUP BY e HAVING. Quando tais padrões são detectados, o sistema desabilita a adição automática de cláusulas LIMIT, evitando interferir em consultas cujo objetivo é sumarizar resultados e não retornar linhas brutas.

Somente após a aprovação nessas etapas a consulta é executada. Os resultados retornados incluem o conjunto de linhas, o tempo de execução e a quantidade de registros, informações que depois são apresentadas na interface e utilizadas na análise das métricas de desempenho. Em caso de falha, o erro é tratado em múltiplos níveis: na inferência, falhas de validação podem ser reencaminhadas ao modelo com *diffs* sintáticos e contexto de erro, inspirando-se no *execution feedback* do SQLGenie; no backend, mensagens de erro incluem detalhes técnicos e orientações resumidas para correção. A Figura 7 exemplifica uma execução incorreta tratada com explicação textual.

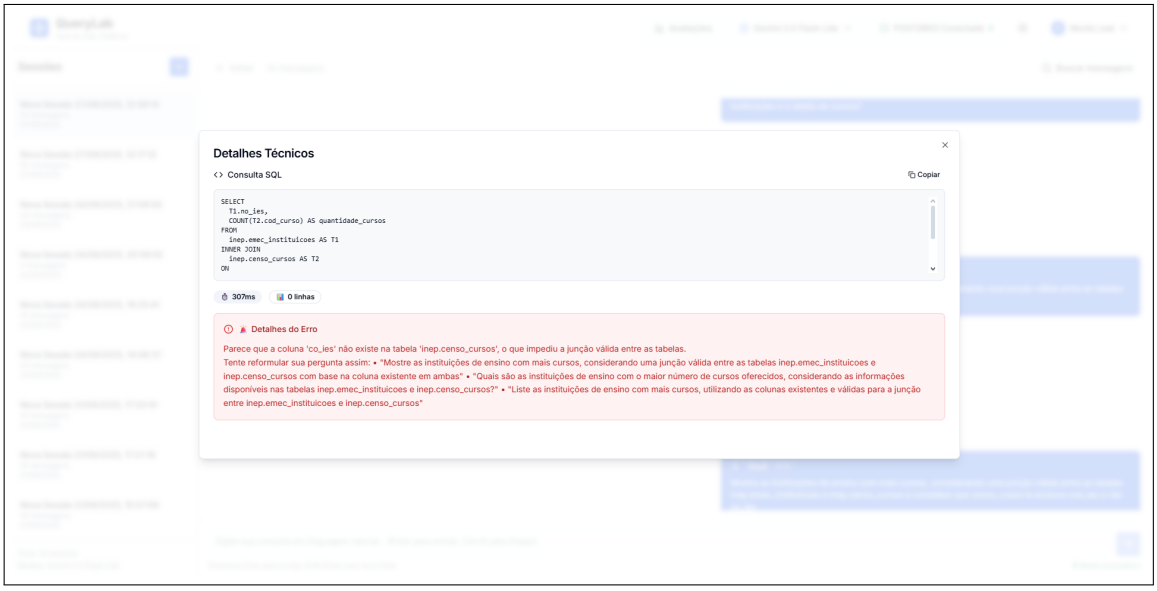

Figura 7 – Erro de execução com diagnóstico automático e sugestão de refinamento.

3.7 Avaliação, métricas e coleta de dados

A avaliação da plataforma combinou métricas automáticas, extraídas diretamente do sistema, com métricas qualitativas fornecidas pelos usuários. Entre as métricas automáticas, destacam-se: a taxa de sucesso de execução, que indica a proporção de consultas SQL válidas e executadas sem erro; o tempo total de resposta, medido desde o envio da pergunta até a apresentação do resultado; e uma medida de complexidade da consulta.[3]

No contexto desta pesquisa, a complexidade foi estimada a partir de três fatores principais: (i) a presença de operações de agregação, como COUNT, SUM, AVG, MIN e MAX, que normalmente aumentam o custo computacional; (ii) o número de JOINs utilizados, uma vez que junções elevam a cardinalidade intermediária, ampliam o plano de execução e tornam a consulta estruturalmente mais complexa; e (iii) o número de linhas retornadas pelo banco de dados, empregado como indicador de abrangência da consulta. Assim,

[3] A literatura de benchmarks em SQL considera fatores como número de junções, agregações e cardinalidade intermediária como indicadores de complexidade estrutural e computacional de consultas (Team, 2023; Yu et al., 2018). A métrica adotada neste trabalho segue essa direção, adaptada ao contexto de avaliação de modelos NL2SQL.

consultas com múltiplas junções, agregações extensivas ou retorno volumoso de registros foram consideradas mais complexas, enquanto consultas com filtros específicos, poucas relações e agregações simples apresentaram menor complexidade.

Além dessas métricas, o sistema registra também a distribuição de erros por modelo, tipo de falha (sintática, semântica, de permissão, de tempo) e consulta. Esses dados são utilizados tanto para análise comparativa entre os modelos quanto para identificar padrões recorrentes de problemas que possam orientar melhorias futuras na orquestração ou nas instruções fornecidas aos modelos.

Complementarmente às métricas automáticas, foi realizado um processo de avaliação com usuários. O convite para participação foi enviado por canais institucionais da UFSM, em especial para as listas `matriculados-cc@inf.ufsm.br` e `matriculados-si@inf.ufsm.br`, que concentram estudantes de Ciência da Computação e Sistemas de Informação. Também houve divulgação junto à PROPLAN da UFSM, envolvendo profissionais que atuam com dados e planejamento institucional.

Os participantes acessaram a plataforma, formularam perguntas sobre a base educacional e preencheram um formulário de avaliação ao término do uso. O formulário incluía questões sobre facilidade de uso, clareza das respostas, percepção de utilidade, modelo preferido e principais dificuldades encontradas. A interface de avaliação agregada é ilustrada na Figura 8.

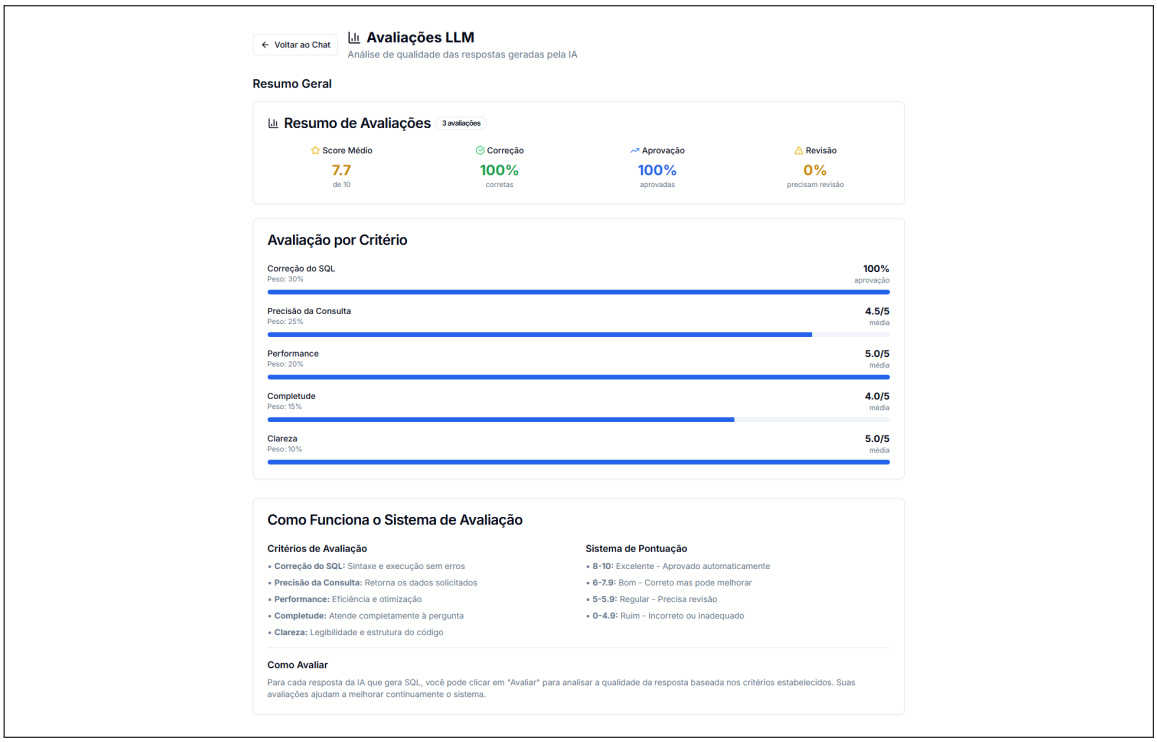

Figura 8 – Tela de avaliação geral, com consolidação de métricas objetivas e qualitativas coletadas durante o uso.

As respostas foram consolidadas e utilizadas para comparar o desempenho relativo dos modelos, identificar padrões de erro e orientar propostas de melhoria para o sistema.

3.8 Síntese da metodologia

Em síntese, a metodologia adotada neste trabalho combina projeto arquitetural em camadas, implementação de um mecanismo de geração paralela de SQL em múltiplos modelos de linguagem, mecanismos de validação e segurança para execução de consultas em um banco de dados real, e uma etapa de avaliação com usuários em contexto acadêmico. Esse conjunto de práticas permite que a plataforma funcione como um laboratório de comparação entre modelos, fornecendo evidências tanto quantitativas quanto qualitativas sobre o comportamento das soluções analisadas e respondendo diretamente aos requisitos e lacunas identificados na fundamentação teórica.

4 RESULTADOS E AVALIAÇÃO

Este capítulo apresenta os resultados obtidos durante a utilização experimental do *QueryLab*. As análises aqui descritas combinam métricas objetivas extraídas automaticamente do sistema, observações estruturais sobre as consultas geradas e percepções qualitativas coletadas por meio de um formulário aplicado aos participantes. Os dados refletem exclusivamente o uso real da plataforma entre os dias 10 e 17 de novembro de 2025, período no qual a aplicação foi utilizada de forma espontânea por diferentes usuários, sem intervenções diretas do autor durante as sessões.

A avaliação teve como propósito verificar se a arquitetura paralela era capaz de aumentar a robustez do sistema, se a representação detalhada do esquema contribuía para a produção de consultas consistentes e se os elementos de explicação apresentados na interface favoreciam a compreensão e a confiança do usuário em relação às respostas obtidas. Esses objetivos orientaram tanto a coleta automática de métricas quanto o processo de avaliação qualitativa.

4.1 Objetivos de avaliação e hipóteses

As hipóteses investigadas ao longo dos experimentos foram construídas a partir da literatura revisada no Capítulo 2 e da arquitetura proposta na Seção 3. A primeira hipótese previa que a execução paralela dos modelos reduziria a probabilidade de falha total do sistema, ao oferecer múltiplas hipóteses de SQL para uma mesma pergunta. A segunda hipótese relacionava-se à expectativa de que a serialização detalhada do esquema, incluindo chaves, colunas e relações frequentes, favoreceria a geração de consultas corretas, especialmente em cenários que envolvem múltiplas tabelas e junções mais complexas. Por fim, a terceira hipótese considerava que a apresentação comparativa dos resultados, com explicações em linguagem natural e indicadores de execução, poderia aumentar a clareza interpretativa e a confiança do usuário.

A seguir, são apresentados os dados que serviram de base empírica para a análise dessas três hipóteses, bem como as implicações observadas a partir da comparação entre os modelos utilizados.

4.2 Dados utilizados e caracterização do esquema

A base de dados empregada nos experimentos corresponde ao esquema real do INEP, que reúne informações institucionais, cursos, municípios, matrículas e indicadores educacionais em larga escala. Embora o conjunto completo ultrapasse noventa tabelas,

apenas parte desse universo foi de fato serializada e utilizada pelos modelos. Cerca de cinquenta tabelas foram inspecionadas por meio de um script de extração de metadados, reunindo nomes de colunas, tipos, tamanhos, chaves declaradas e relações estruturais. Esse processo resultou em um subconjunto representativo das estruturas mais frequentes durante o uso da plataforma, preservando a heterogeneidade terminológica e a complexidade típica do domínio.

A documentação técnica completa do esquema, incluindo trechos do script utilizado na varredura, amostras do arquivo de metadados condensado e exemplos do JSON integral que descreve tabelas, colunas e chaves primárias, encontra-se reunida no Apêndice A. Esse material complementa o arquivo `inep-schema-docs.md`, que apresenta a descrição detalhada das entidades processadas, e reproduz exatamente a mesma estrutura disponibilizada aos modelos durante a geração das consultas. A inclusão desses documentos no apêndice reforça a reprodutibilidade do experimento, permitindo que outros pesquisadores reconstruam integralmente o ambiente utilizado.

A escolha desse domínio mostrou-se adequada ao propósito do estudo, pois os dados do INEP envolvem relações não triviais entre entidades, variações semânticas significativas e tabelas de grande porte, características que desafiam os modelos a interpretar corretamente chaves, vínculos e padrões de junção. Esse cenário contribuiu para uma avaliação mais realista da capacidade dos modelos de linguagem em navegar em esquemas relacionais amplos e heterogêneos.

4.3 Ambiente experimental

O *QueryLab* foi executado em um ambiente composto por dois bancos PostgreSQL independentes. O primeiro, exclusivo para leitura, armazenava o esquema e os dados do INEP utilizados nos experimentos. O segundo banco, de caráter operacional, registrava sessões, consultas enviadas, hipóteses geradas pelos modelos, métricas de execução e avaliações qualitativas dos usuários. O backend, implementado em Node.js, realizava a orquestração entre o banco de dados e três provedores distintos de modelos de linguagem: Gemini, Groq e SQLCoder.

No caso do Gemini, foram utilizadas duas versões gratuitas disponíveis na data dos experimentos: *Gemini 2.5 Flash Free*, com limite aproximado de 1.000 requisições diárias, e *Gemini 2.0 Flash*, configurado como mecanismo de *fallback* para situações em que o limite diário da versão principal fosse atingido. Para o Groq, empregou-se a variante *Llama 3.3 70B Versatile*, que combina baixa latência com boa capacidade de raciocínio tabular. O SQLCoder foi acessado por meio do provedor *Cloudflare Workers AI*, igualmente em sua modalidade gratuita.

Todos os modelos receberam, em princípio, o mesmo contexto de esquema, composto por nomes de tabelas, colunas, chaves primárias e estrangeiras, além de descrições auxiliares. Assim, as diferenças observadas entre as consultas geradas refletem as características intrínsecas de cada modelo, como capacidade de *schema linking*, robustez sintática e mecanismos internos de geração, e não variações no material informacional fornecido.

Uma exceção relevante ocorre no caso do SQLCoder. O provedor utilizado impõe um limite de tokens de entrada significativamente menor em comparação aos serviços do Gemini e do Groq, o que exigiu uma *redução inteligente e seletiva* do prompt. Esse processo envolveu condensar descrições, remover redundâncias e priorizar apenas as partes

essenciais do esquema para cada pergunta, garantindo que o modelo permanecesse dentro do limite de contexto sem perda relevante de informação. Ainda assim, manteve-se a equivalência estrutural do esquema fornecido, de modo a preservar a comparabilidade entre os modelos.

Finalmente, cabe registrar uma limitação experimental relevante: o presente trabalho não incluiu modelos *specialized* de text-to-SQL, como RAT-SQL, PICARD ou DIN-SQL, amplamente utilizados como baselines tradicionais na literatura. Embora tais modelos pudessem fornecer uma comparação mais ampla, sua utilização mostrou-se inviável no escopo deste projeto. A pesquisa optou exclusivamente por modelos gratuitos, acessíveis e executados via API, de modo a garantir reprodutibilidade e evitar dependência de GPUs locais. Tentou-se também utilizar o serviço *Replicate* para executar versões hospedadas desses modelos especializados, porém os *cold starts* frequentes em modelos menos populares ou privados resultaram em latências elevadas e falhas recorrentes de inicialização, inviabilizando seu uso na plataforma. Diante disso, a avaliação concentrou-se nos modelos generalistas disponíveis gratuitamente, complementados pelo SQLCoder, registrando-se essa limitação como parte das restrições experimentais do estudo.

4.4 Métricas e instrumentos

A avaliação objetiva baseou-se em métricas coletadas automaticamente ao longo do período experimental, compreendido entre 10 e 17 de novembro de 2025. No total, foram geradas 32 consultas, das quais 27 foram executadas com sucesso, resultando em uma taxa geral de sucesso de 84,38%. Não foram registradas falhas de execução, mas algumas hipóteses de SQL produzidas pelos modelos foram descartadas antes da execução por não atenderem aos critérios mínimos de validade estrutural.

O tempo médio de execução das consultas foi de 710,69 ms, com mediana de 629 ms, mínimo de 120 ms, percentil 95 de 1152 ms e tempo máximo de 2354 ms. Esses valores indicam desempenho estável, com apenas três consultas ultrapassando um segundo de execução. A média de linhas retornadas foi de 16,31 por consulta, com um máximo de 100 linhas, refletindo características de consultas reais com agregações, junções e filtragens moderadas.

Quanto à estrutura das consultas, observou-se que a maioria apresentou graus relevantes de complexidade: 29 das 32 consultas incluíram junções explícitas, 27 continham cláusulas `WHERE`, 14 apresentaram `GROUP BY`, 13 utilizaram `ORDER BY` e 25 empregaram limites explícitos de resultados (`LIMIT`). O tamanho médio das consultas geradas foi de 390,66 caracteres, variando entre 108 e 816 caracteres, o que evidencia a diversidade estrutural e a necessidade de compreender relações entre múltiplas tabelas.

No agregado, as métricas diárias revelam flutuações naturais no uso, coerentes com a natureza exploratória do experimento. Houve dias com maior atividade, como 12 de novembro (oito consultas), e outros com menor volume, como 10 e 17 de novembro (três consultas cada). No total, foram registradas doze sessões únicas de utilização, com variação entre uma e seis consultas por sessão, conduzidas por diferentes usuários e com taxas de sucesso entre 50% e 100%.

4.5 Procedimento

O procedimento experimental seguiu o fluxo descrito na Seção 3. A partir de cada pergunta submetida pelo usuário, os modelos de linguagem eram acionados em paralelo, recebendo idêntico contexto de esquema. Cada hipótese de SQL era submetida a um processo de normalização, validação sintática e inspeção de segurança antes da execução no banco de dados read-only. As respostas eram exibidas lado a lado, possibilitando comparação direta entre os modelos.

Não houve roteiros pré-definidos, e nenhuma intervenção adicional foi realizada durante as sessões, de modo que a variabilidade das perguntas e a diversidade das estruturas SQL refletiram o comportamento espontâneo dos participantes. Esse desenho experimental aproxima o ambiente testado de cenários de uso reais e reduz vieses associados a benchmarks artificiais.

4.6 Resultados

Os resultados indicam que a plataforma apresentou desempenho consistente ao longo do período analisado. A ausência de erros de execução confirma a eficácia das camadas de validação e proteção do backend, que impediram que consultas malformadas fossem enviadas ao banco de dados. A alta proporção de junções e cláusulas estruturais evidencia que os usuários formularam perguntas que exigiam compreensão de relacionamentos complexos entre entidades.

Entre as 32 consultas analisadas, 29 envolveram junções explícitas, 27 apresentaram filtros, 14 utilizaram agregações com `GROUP BY`, 13 aplicaram ordenações e 16 recorreram a `DISTINCT`. Esse comportamento demonstra que a plataforma foi utilizada em cenários que exigem raciocínio relacional, indo além de consultas simples. Em termos de desempenho, apenas três consultas excederam um segundo, e nenhuma ultrapassou cinco segundos de execução, o que está em conformidade com o custo esperado de operações moderadas sobre tabelas relacionais de grande porte.

4.7 Avaliação qualitativa via formulário

A avaliação qualitativa foi realizada com quatro participantes que utilizaram a plataforma durante o período experimental. Em relação à familiaridade com dados da educação superior, dois participantes declararam nível intermediário, um indicou nível básico e um afirmou não possuir conhecimento prévio nesse tipo de dado. Já quanto a Inteligência Artificial e modelos de linguagem, três participantes relataram conhecimento intermediário e um indicou nível básico. Metade dos respondentes já havia utilizado anteriormente ferramentas que permitem realizar consultas em linguagem natural sobre bases de dados, enquanto a outra metade não possuía experiência prévia com esse tipo de solução.

De modo geral, as avaliações foram positivas. A interface foi descrita como clara e fácil de usar, e o envio de consultas em linguagem natural foi considerado simples e intuitivo pela maior parte dos participantes. A apresentação dos resultados também recebeu avaliações favoráveis, com comentários destacando que as informações retornadas eram fáceis de compreender. As explicações em linguagem natural exibidas junto às consultas SQL foram percebidas como úteis, sobretudo para entender filtros, junções e critérios de agregação empregados nas consultas geradas.

O modo comparativo, que apresenta lado a lado as respostas dos diferentes modelos (Gemini, Groq e SQLCoder), foi apontado como um dos elementos mais relevantes da plataforma. Os participantes relataram que essa visualização paralela facilitou a compreensão das diferenças entre os modelos e contribuiu para aumentar a confiança na interpretação dos resultados. Em termos de preferência, dois respondentes indicaram o Gemini como modelo mais satisfatório na maioria das consultas, um destacou o SQLCoder e outro observou que a escolha dependia do tipo de pergunta realizada.

As respostas abertas trouxeram sugestões específicas de melhoria. Dois participantes mencionaram a importância de disponibilizar, de forma mais acessível, listagens de atributos e tabelas do esquema, de modo a facilitar a formulação de consultas mais precisas. Também foram apontadas dúvidas quanto à diferença entre áreas da interface e ao modo desenvolvedor, bem como o desejo de editar e testar a consulta SQL diretamente antes de copiá-la. Outro comentário ressaltou a dificuldade de formular perguntas sem conhecer bem a estrutura da base de dados, sugerindo mecanismos adicionais de apoio ao usuário. Ainda assim, todos afirmaram que utilizariam novamente a ferramenta em contextos de estudo ou trabalho, e três indicaram que a recomendariam a outras pessoas, reforçando a percepção de utilidade e potencial da plataforma.

4.8 Discussão

A análise dos resultados permite avaliar as hipóteses formuladas inicialmente. A execução paralela demonstrou reduzir o impacto de falhas isoladas na geração de SQL, na medida em que ao menos um dos modelos produziu consultas válidas na maioria dos casos. A serialização detalhada do esquema contribuiu para a geração correta de consultas que envolviam múltiplas tabelas, reforçando a hipótese de que a contextualização explícita favorece a qualidade estrutural do SQL.

O componente explicativo desempenhou papel importante na experiência dos usuários, permitindo maior compreensão dos filtros, agregações e relações estruturais representadas nas consultas SQL. Assim, a hipótese de que a explicabilidade contribuiria para maior confiança e clareza foi corroborada pelos relatos.

Algumas limitações permanecem. A amostra reduzida de participantes limita a generalização dos resultados, e a natureza espontânea das perguntas implica heterogeneidade na complexidade das consultas. Além disso, o esquema heterogêneo do INEP pode introduzir ambiguidades interpretativas que variam de acordo com o modelo utilizado, dificultando comparações estritamente controladas. Apesar disso, os resultados obtidos constituem evidência consistente da viabilidade e utilidade da abordagem proposta para fins de experimentação e análise comparativa.

5 CONCLUSÃO

Este trabalho apresentou o *QueryLab*, uma plataforma web que transforma perguntas formuladas em linguagem natural em consultas SQL executáveis sobre um banco de dados real, combinando técnicas de *schema serialization*, execução paralela entre modelos de linguagem e mecanismos de segurança voltados a preservar a integridade do sistema. A solução foi projetada como um ambiente de experimentação e análise comparativa, capaz de expor divergências entre modelos, registrar métricas detalhadas de execução e apoiar

o usuário na interpretação dos resultados por meio de explicações textuais, indicadores técnicos e amostras de dados.

Ao longo do desenvolvimento e avaliação, o *QueryLab* demonstrou que é possível aproximar sistemas *text-to-SQL* de cenários reais de uso sem recorrer a benchmarks artificiais ou conjuntos de perguntas rigidamente controlados. As sessões realizadas com usuários mostraram que o sistema é capaz de produzir SQL corretas em grande parte das interações, com uma taxa de sucesso superior a 84% entre as consultas que alcançaram a etapa de execução, e com tempos de resposta compatíveis com aplicações exploratórias sobre bancos relacionais heterogêneos. O modo comparativo, por sua vez, evidenciou claras diferenças de estilo, latência e completude entre os modelos avaliados, cumprindo o papel de tornar a inferência mais transparente e interpretável.

As principais contribuições deste trabalho concentram-se em quatro eixos. O primeiro é a arquitetura de orquestração multi-LLM, que reduz pontos de falha e fornece visibilidade granular do comportamento de cada modelo. O segundo é o pipeline explicável que combina SQL gerada, amostra de resultados, metadados técnicos e explicações em linguagem natural, permitindo que usuários sem formação especializada compreendam a lógica das respostas. O terceiro é o conjunto de mecanismos de segurança, entre eles execução em modo *read-only*, validação sintática, auditoria e limites automáticos, que viabilizam a interação segura entre modelos generativos e bancos reais. O quarto é a metodologia de avaliação integrando métricas objetivas (execução, latência, complexidade das consultas) e julgamento humano estruturado, oferecendo uma visão mais completa do comportamento dos modelos e do valor agregado pela plataforma.

Como toda solução em estágio inicial, o *QueryLab* apresenta limitações importantes. A dependência de serviços externos torna o desempenho sensível a latências de rede e flutuações nos provedores. O custo de uso cresce de forma proporcional ao volume de tokens, o que demanda estratégias de otimização contínua. A ausência de suporte a bases não relacionais ou esquemas altamente dinâmicos restringe o escopo atual da ferramenta. Além disso, a heterogeneidade do esquema do INEP, embora adequada para testes realistas, introduz ambiguidades que podem ocasionar hipóteses de SQL incompletas ou incorretas em consultas mais complexas.

Ainda assim, essas limitações apontam caminhos naturais para evolução da plataforma. Trabalhos futuros incluem a integração de mecanismos de reparo automático orientados por custo e por políticas de *timeout*, a incorporação de camadas semânticas adicionais ao *schema* (como dicionários de sinonímia, ontologias e descrições textuais enriquecidas), e a extração de planos de execução via `EXPLAIN` para fins didáticos ou diagnósticos. A ampliação dos experimentos com especialistas da área, bem como a realização de testes controlados com diferentes modelos de linguagem, poderá oferecer resultados mais abrangentes e comparáveis. Por fim, a expansão da plataforma para novos domínios de dados públicos e a inclusão de suporte multilíngue representam passos importantes em direção ao uso generalizado do *QueryLab*.

Em síntese, o *QueryLab* contribui para o avanço de sistemas *text-to-SQL* ao oferecer um ambiente explicável, comparativo e próximo de condições reais de uso, demonstrando que a combinação de contexto estruturado, execução paralela e transparência nas respostas pode ampliar significativamente a utilidade prática desses sistemas. Os resultados obtidos, ainda que delimitados pelo escopo experimental, reforçam o potencial da abordagem e abrem espaço para novas investigações sobre modelos de linguagem aplicados à consulta e

compreensão de dados estruturados.

REFERÊNCIAS

ASSUMPçãO, J. *Análise Comparativa de Modelos de Linguagem para a Tarefa Text-to-SQL em Bases Educacionais do INEP*. Dissertação (Trabalho de Conclusão de Curso (Engenharia de Computação)) — Universidade Federal de Santa Maria (UFSM), Santa Maria, RS, Brasil, 2025. Manuscrito cedido pelo autor. Citado 2 vezes nas páginas 14 e 23.

DENG, N.; CHEN, Y.; ZHANG, Y. Recent advances in text-to-sql: A survey of what we have and what we expect. In: *Proceedings of the 29th International Conference on Computational Linguistics (COLING)*. Gyeongju, Republic of Korea: [s.n.], 2022. Disponível em: <https://aclanthology.org/2022.coling-1.190.pdf>. Citado 2 vezes nas páginas 13 e 15.

GAO, D. et al. Text-to-sql empowered by large language models: A benchmark evaluation. In: *Proceedings of the VLDB Endowment (PVLDB)*. [s.n.], 2024. v. 17, n. 5, p. 1132–1145. Disponível em: <https://www.vldb.org/pvldb/vol17/p1132-gao.pdf>. Citado 5 vezes nas páginas 15, 18, 19, 20 e 27.

GHOSH, P.; JAIN, A.; YENIGALLA, P. Sqlgenie: A practical llm-based system for reliable and efficient sql generation. In: *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Industry Track)*. [s.n.], 2025. Disponível em: <https://aclanthology.org/2025.acl-industry.71.pdf>. Citado 5 vezes nas páginas 15, 18, 19, 27 e 29.

HONG, Z. et al. A survey of text-to-sql in the era of llms. *IEEE Transactions on Knowledge and Data Engineering*, 2025. Early access / in press. Disponível em: <https://dbgroup.cs.tsinghua.edu.cn/ligl/papers/TKDE25-NL2SQL.pdf>. Citado 3 vezes nas páginas 15, 18 e 20.

HONG, Z. et al. Next-generation database interfaces: A survey of llm-based text-to-sql. *arXiv preprint arXiv:2406.08426*, 2024. Disponível em: <https://arxiv.org/abs/2406.08426>. Citado 2 vezes nas páginas 15 e 18.

LAB, Y. L.; SALESFORCE. *SParC: Semantic Parsing in Context (Text-to-SQL multi-turn)*. 2019. Website. Disponível em: <https://yale-lily.github.io/sparc>. Citado na página 20.

QUAMAR, A. et al. Natural language interfaces to data. *Foundations and Trends in Databases*, Now Publishers, v. 11, n. 1, p. 1–131, 2022. Disponível em: <https://www.nowpublishers.com/article/DownloadSummary/DBS-078>. Citado 3 vezes nas páginas 13, 14 e 15.

SCHOLAK, T.; SCHUCHER, N.; BAHDANAU, D. Picard: Parsing incrementally for constrained auto-regressive decoding from language models. *arXiv:2109.05093*, 2021. Disponível em: <https://arxiv.org/abs/2109.05093>. Citado 4 vezes nas páginas 13, 15, 19 e 27.

TEAM, B. *LiveSQLBench (BIRD-SQL Pro): A contamination-free, evolving text-to-SQL benchmark*. 2024. Website. Disponível em: <https://livesqlbench.ai/>. Citado 3 vezes nas páginas 13, 21 e 22.

TEAM, B. B. *BIRD: A Text-to-Efficient-SQL Benchmark*. 2023. Website. Disponível em: <https://bird-bench.github.io/>. Citado 3 vezes nas páginas 21, 22 e 28.

TREVISAN, M. *Geração de Consultas SQL a partir de Linguagem Natural com Modelos de Aprendizado de Máquina*. Dissertação (Trabalho de Conclusão de Curso (Engenharia de Computação)) — Universidade Federal de Santa Maria (UFSM), Santa Maria, RS, Brasil, 2024. Disponível no Repositório Institucional da UFSM. Acesso em: 21 out. 2025. Disponível em: <https://repositorio.ufsm.br/handle/1/33876>. Citado 2 vezes nas páginas 14 e 23.

WANG, B. et al. Rat-SQL: Relation-aware schema encoding and linking for text-to-SQL parsers. In: *ACL*. [s.n.], 2020. Disponível em: <https://aclanthology.org/2020.acl-main.677/>. Citado 2 vezes nas páginas 15 e 19.

XIE, W.; DAI, Y.; JIANG, W. Sde-sql: Enhancing text-to-sql generation in large language models via self-driven exploration with sql probes. *arXiv preprint arXiv:2506.07245*, 2025. Disponível em: <https://arxiv.org/abs/2506.07245>. Citado 2 vezes nas páginas 19 e 29.

XU, X.; LIU, C.; SONG, D. Sqlnet: Generating structured queries from natural language without reinforcement learning. In: *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. [s.n.], 2017. p. 2380–2389. Disponível em: <https://aclanthology.org/D17-1259/>. Citado na página 15.

YU, T. et al. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task. In: *EMNLP*. [s.n.], 2018. Disponível em: <https://aclanthology.org/D18-1425/>. Citado 3 vezes nas páginas 13, 20 e 22.

YU, T. et al. Cosql: A conversational text-to-SQL challenge towards cross-domain natural language interfaces to databases. In: *EMNLP-IJCNLP*. [s.n.], 2019. Disponível em: <https://aclanthology.org/D19-1204/>. Citado 3 vezes nas páginas 13, 20 e 22.

ZAIDI, E. et al. *Predicts 2025: 4 Ways AI Will Disrupt Data Management Markets and Solutions*. 2024. Blog post summarizing Gartner report. Relatório Gartner "Predicts 2025". Disponível em: <https://www.enterprisedb.com/blog/predicts-2025-4-ways-ai-will-disrupt-data-management-markets-and-solutions>. Citado 2 vezes nas páginas 13 e 14.

ZHANG, e. a. Understanding the effects of noise in text-to-sql benchmarks: A case study on bird-bench. *arXiv preprint arXiv:2402.12243*, 2024. Disponível em: <https://arxiv.org/abs/2402.12243>. Citado 3 vezes nas páginas 13, 21 e 22.

ZHONG, V.; XIONG, C.; SOCHER, R. Seq2sql: Generating structured queries from natural language using reinforcement learning. *arXiv:1709.00103*, 2017. Disponível em: <https://arxiv.org/abs/1709.00103>. Citado na página 15.

APÊNDICE A – DOCUMENTAÇÃO TÉCNICA DO ESQUEMA INEP E DO PROCESSO DE DESCOBERTA

Este apêndice apresenta uma síntese do processo de extração e documentação do esquema do INEP utilizado nos experimentos conduzidos pelo *QueryLab*. A plataforma depende de uma representação estruturada do banco de dados para que os modelos possam identificar relações, atributos e entidades relevantes durante a geração das consultas em linguagem SQL. Por esse motivo, foi desenvolvido um procedimento automatizado de descoberta de esquema que percorre as tabelas do ambiente PostgreSQL associado ao domínio educacional e produz arquivos técnicos consumidos pelo backend. A versão completa de todos os arquivos apresentados nesta seção, incluindo código original, documentação integral e estrutura completa das cinquenta tabelas processadas, está disponível no repositório oficial do projeto, em `https://github.com/muriloleal13/querylab`.

1 SCRIPT DE DESCOBERTA DO ESQUEMA

A descoberta do esquema foi realizada por meio de um script em JavaScript que se conecta ao banco de dados, consulta as tabelas declaradas no schema `inep` e extrai informações sobre colunas, tipos, nulabilidade e chaves primárias. O trecho abaixo ilustra a estrutura central do código utilizado para essa finalidade, preservando o formato e a lógica do arquivo original:

```
const tablesQuery = `
  SELECT tablename as table_name
  FROM pg_catalog.pg_tables
  WHERE schemaname = 'inep'
    AND tablename = ANY($1::text[])
  ORDER BY tablename
`;

const columnsQuery = `
  SELECT column_name, data_type, is_nullable
  FROM information_schema.columns
  WHERE table_schema = 'inep' AND table_name = $1
`;

const primaryKeysQuery = `
  SELECT a.attname AS column_name
  FROM pg_class c
  JOIN pg_index i ON i.indrelid = c.oid
  JOIN pg_attribute a ON a.attrelid = c.oid
                      AND a.attnum = ANY(i.indkey)
  WHERE c.relname = $1 AND i.indisprimary
`;
```

O script percorre a lista de tabelas definidas como relevantes e, para cada uma delas, monta objetos contendo todas as colunas encontradas, seus tipos e as chaves primárias identificadas no catálogo do PostgreSQL. Ao final, três arquivos são gerados: um documento markdown com a listagem legível, um arquivo JSON com a versão integral do esquema e uma versão condensada destinada ao consumo pelos modelos.

2 RESUMO CONDENSADO DO ESQUEMA

O arquivo `inep-schema-summary.json` contém uma versão reduzida e otimizada para uso como contexto durante a geração de SQL. Esse formato remove elementos supérfluos e representa cada coluna no formato compacto `nome:tipo`. A seguir apresenta-se um trecho ilustrativo do resumo utilizado:

```
{
  "schemaName": "inep",
  "totalTables": 50,
  "tables": [
    {
      "name": "censo_ies",
      "columns": [
        "cod_ies:int",
        "nm_ies:varchar",
        "tp_categoria_administrativa:int",
        "sg_uf_ies:varchar"
      ]
    },
    {
      "name": "matriculas_municipio",
      "columns": [
        "nu_ano_censo:int",
        "co_municipio:int",
        "qt_ing_18_24:int",
        "qt_mat_total:int"
      ]
    }
  ]
}
```

O arquivo completo inclui todas as cinquenta tabelas processadas e foi utilizado pelo backend sempre que um modelo precisava interpretar estruturas e relações básicas entre entidades.

3 DOCUMENTAÇÃO TEXTUAL DO ESQUEMA

O arquivo `inep-schema-docs.md` apresenta uma descrição textual mais detalhada de cada tabela do esquema, incluindo todas as colunas e seus tipos. O trecho a seguir exemplifica a forma como as tabelas são apresentadas no documento:

```
### Tabela: censo_categorias_administrativas
- id_categoria_administrativa: integer (PRIMARY KEY)
- descr_categoria_administrativa: varchar
- descr_cat_adm_agrupada: varchar

### Tabela: municipios_ibge
- cod_municipio: integer (PRIMARY KEY)
- nome_municipio: varchar
- cod_ibge: integer
- nome_regiao: varchar
```

Essa documentação foi utilizada durante o desenvolvimento para validar eventuais inconsistências nas consultas geradas pelos modelos e serviu como referência manual para inspeções realizadas fora da plataforma. A listagem integral contém a totalidade das tabelas processadas, mas apenas trechos representativos foram incluídos aqui devido ao seu tamanho.

4 ESTRUTURA COMPLETA DE METADADOS

O arquivo `inep-schema-full.json` constitui a forma mais detalhada da representação do esquema. Além dos nomes das colunas e tipos, também registra informações sobre nulabilidade, chaves primárias e data de descoberta. O trecho a seguir demonstra sua organização:


```
{
  "schemaName": "inep",
  "discoveredAt": "2025-11-06T22:57:27.590Z",
  "tables": [
    {
      "name": "capes_cursos_bruto",
      "columns": [
        { "name": "an_base", "dataType": "integer", "nullable": false
        ↪  },
        { "name": "nm_area_conhecimento", "dataType": "varchar",
        ↪  "nullable": true },
        { "name": "nm_entidade_ensino", "dataType": "varchar",
        ↪  "nullable": true }
      ],
      "primaryKeys": []
    }
  ]
}
```


A versão integral desse arquivo foi usada pelo backend durante a etapa de validação semântica das consultas, permitindo identificar usos incorretos de colunas, tabelas inexistentes ou inconsistências estruturais nas hipóteses produzidas pelos modelos.

5 DISPONIBILIZAÇÃO INTEGRAL DOS ARQUIVOS

Todos os documentos mencionados — `inep-schema-full.json`, `inep-schema-summary.json`, `inep-schema-docs.md`, bem como o script

`discover-schema.js` — estão disponíveis de forma completa, original e atualizada no repositório oficial do projeto. A disponibilização pública desses artefatos garante reprodutibilidade, transparência e a possibilidade de que outros pesquisadores reconstruam o ambiente experimental utilizado neste trabalho.

APÊNDICE B – ALGORITMOS AUXILIARES DO *QUERYLAB*

Algorithm 2: GerarSQLMultiModelo($q, schema, hints$)

Entrada: Pergunta q; esquema alvo serializado; conjunto de instruções auxiliares $hints$

Saída: $draftSQLs = \{sql_{gemini}, sql_{groq}, sql_{sqlcoder}\}$

1 $q_{norm} \leftarrow$ NormalizarPergunta(q)

2 $ctx \leftarrow$ {schema serializado, exemplos relevantes, palavras-chave do domínio, $hints$}

3 Iniciar **três** chamadas assíncronas, cada uma com (q_{norm}, ctx):

- $T_{gemini} \leftarrow$ ChamarGemini(q_{norm}, ctx)
- $T_{groq} \leftarrow$ ChamarGroq(q_{norm}, ctx)
- $T_{sqlcoder} \leftarrow$ ChamarSQLCoder(q_{norm}, ctx)

Aguardar todos os futuros com timeout global T_{global}

if *qualquer chamada exceder* T_{global} **then**

  Marcar hipótese correspondente como *timeout*

Para cada resposta recebida:

- Extrair apenas o trecho SQL bruto
- Remover explicações, formatações e comentários
- Aplicar normalização sintática (NormalizarSQL)

$draftSQLs \leftarrow \{\text{sql}_{gemini}, \text{sql}_{groq}, \text{sql}_{sqlcoder}\}$

retorna $draftSQLs$

Algorithm 3: ValidarEExecutar($sql, schema, constraints$)

Entrada: SQL candidato; esquema alvo; políticas de segurança e limites
Saída: $result = \{status, rows, metrics, sqlNorm, erro\}$

1 $sqlNorm \leftarrow$ NormalizarSQL(sql)
2 **if** *ContemComandosProibidos*($sqlNorm$) **then**
3 **retorna** {FALHA_SQL_BLOQUEIO, `[]`, `{}`, sqlNorm, "comando proibido"}
4 **if** *EhMultiplaInstrucao*($sqlNorm$) **then**
5 **retorna** {FALHA_SQL_BLOQUEIO, `[]`, `{}`, sqlNorm, "múltiplas instruções"}
6 **if** *ParserSQL*($sqlNorm$) *falhar* **then**
7 **retorna** {FALHA_SQL_PARSE, `[]`, `{}`, sqlNorm, "erro de sintaxe"}
8 **if** *ReferenciasInvalidas*($sqlNorm, schema$) **then**
9 **retorna** {FALHA_SQL_SCHEMA, `[]`, `{}`, sqlNorm, "tabela/coluna inválida"}
10 $sqlSafe \leftarrow$ AplicarLimitETimeout($sqlNorm, constraints$)
11 Aplicar filtros RLS e escopo por *tenant* conforme $constraints$
12 Iniciar cronômetro $t_0 \leftarrow$ Agora()
13 $rows \leftarrow$ ExecutarSomenteLeitura($sqlSafe$)
14 **retorna** {FALHA_SQL_EXEC, `[]`, `{}`, sqlNorm, "erro de execução"}
15 $t_1 \leftarrow$ Agora()
16 $tempoExec \leftarrow t_1 - t_0$
17 $nLinhas \leftarrow$ Comprimento($rows$)
18 $preview \leftarrow$ primeiras $N_{maxPreview}$ linhas de $rows$
19 **if** $constraints.coletarPlano = verdadeiro$ **then**
20 $plano \leftarrow$ ExplainLimitado($sqlSafe$)
21 **else**
22 $plano \leftarrow$ `null`
23 $metrics \leftarrow$ { tempoExec, nLinhas, preview, plano}
24 RegistrarTelemetria($sqlNorm, schema, metrics, constraints$)
25 **retorna** {SUCESSO, rows, metrics, sqlNorm, `null`}